

**МИНОБРНАУКИ РОССИИ**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ**  
**УНИВЕРСИТЕТ им. В.Г. ШУХОВА»**  
**(БГТУ им. В.Г. Шухова)**

**Институт** информационных технологий и управляющих систем

**Кафедра** программного обеспечения вычислительной техники и автоматизированных систем

**Направление подготовки** 09.03.04 Программная инженерия

**Направленность (профиль) образовательной программы** Разработка программно-информационных систем

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**

на тему:

**Реализация алгоритма извлечения звука  
музыкального инструмента из звукового потока  
на основе методов искусственного интеллекта**

**Студент (ка):** Барышникова Варвара Дмитриевна  
**Зав. кафедрой:** канд. техн. наук, доц. Поляков Владимир Михайлович  
**Руководитель:** канд. физ.-мат. наук, доц. Осипов Олег Васильевич

**К защите допустить**  
**Зав. кафедрой** \_\_\_\_\_ /Поляков В.М./

« \_\_\_\_\_ » \_\_\_\_\_ 2026 г.

Белгород 2026 г.

**МИНОБРНАУКИ РОССИИ**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ**  
**УНИВЕРСИТЕТ им. В.Г. ШУХОВА»**  
**(БГТУ им. В.Г. Шухова)**

**Институт** информационных технологий и управляющих систем

**Кафедра** программного обеспечения вычислительной техники и автоматизированных систем

**Направление подготовки** 09.03.04 Программная инженерия

**Направленность (профиль) образовательной программы** Разработка программно-информационных систем

Утверждаю:

Зав. кафедрой \_\_\_\_\_

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

**ЗАДАНИЕ**

на выпускную квалификационную работу студента (ки)

*Иванова Ивана Ивановича*

---

1. Вид выпускной квалификационной работы (ВКР): *бакалаврская работа*.
2. Тема ВКР *Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения* утверждена приказом по университету от «28» января 2026 г. №2/124.
3. Срок сдачи студентом (кой) законченной ВКР: «12» июня 2026 г.
4. Исходные данные: ключевые слова, название алгоритмов и методов, технологии разработки и т.п. Пример: рекомендательные системы, подходы построения рекомендательных систем, методы матричной факторизации, оптимизационные методы обучения модели, аффинные преобразования, трёхмерная графика, машинное обучение, клиент-серверное приложение, генеративная нейронная сеть.
5. Содержание ВКР: Здесь должны быть перечислены названия разделов (глав) пояснительной записки. Пример:
  - Введение,
  - Описание предметной области прогнозирования погоды,
  - Разработка архитектуры программного обеспечения прогнозирования погоды,
  - Разработка алгоритмов и программного обеспечения прогнозирования погоды,
  - Тестирование программной системы прогнозирования погоды,
  - Руководство пользователя разработанной системы прогнозирования погоды,
  - Заключение,
  - Список литературы,
  - Приложение А,
  - Приложение Б.

6. Перечень графического материала: 1278 рисунков, 1785 таблиц. Презентация включает: постановку задачи, актуальность, технологии, численные эксперименты, внешний вид программы, заключение.

Консультанты по работе с указанием относящихся к ним разделов:

| Раздел | Консультант | Задание выдал<br>(подпись, дата) | Задание принял<br>(подпись, дата) |
|--------|-------------|----------------------------------|-----------------------------------|
|        |             |                                  |                                   |

Дата выдачи задания: «17» января 2026 г.

\_\_\_\_\_  
(подпись руководителя)

Задание принял (приняла) к исполнению

\_\_\_\_\_  
(подпись студента (ки))

**КАЛЕНДАРНЫЙ ПЛАН**

| № п/п | Наименование этапов работы                                                                                          | Срок выполнения этапов работы | Примечание |
|-------|---------------------------------------------------------------------------------------------------------------------|-------------------------------|------------|
| 1     | Анализ предметной области. Постановка задачи разработки системы прогнозирования погоды методами машинного обучения. | 18.01.26 – 15.02.26           | Выполнено  |
| 2     | Разработка алгоритмов, структур нейросетей и архитектуры системы прогнозирования погоды.                            | 15.02.26 – 20.03.26           | Выполнено  |
| 3     | Разработка программной части системы прогнозирования погоды.                                                        | 20.03.26 – 29.03.26           | Выполнено  |
| 4     | Разработка клиентской части системы прогнозирования погоды.                                                         | 29.03.26 – 29.04.26           | Выполнено  |
| 5     | Тестирование программного обеспечения для прогнозирования погоды.                                                   | 29.04.26 – 17.05.26           | Выполнено  |
| 6     | Оформление пояснительной записки. Подготовка выступления.                                                           | 17.05.26 – 10.06.26           | Выполнено  |
| 5     | Тестирование программного обеспечения для прогнозирования погоды.                                                   | 29.04.26 – 17.05.26           | Выполнено  |
| 6     | Оформление пояснительной записки. Подготовка выступления.                                                           | 17.05.26 – 10.06.26           | Выполнено  |

Студент (ка) \_\_\_\_\_ *Иванов Иван Иванович*  
(подпись) (Ф.И.О.)

Руководитель \_\_\_\_\_ *Осипов Олег Васильевич*  
(подпись) (Ф.И.О.)

## Результаты проверки ЭВ ВКР на заимствование

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Студент (ка) Иванов И.И. ПВ-212 16.06.2026  
(Ф.И.О.) (подпись) (группа) (дата)

Тема ВКР: Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения.

ВКР прошла проверку на объём заимствований.

Итоговая оценка заимствований: **3,1415926535897932384626433832%.**

Работу проверил \_\_\_\_\_ 16.06.2026 Хлопов А.М.  
(подпись) (дата) (Ф.И.О.)

Руководитель ВКР

|                          |            |                 |
|--------------------------|------------|-----------------|
| канд. соц. наук, доцент  | 16.06.2026 | Островский А.М. |
| (уч. степень, должность) | (подпись)  | (Ф.И.О.)        |

## ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

SiSec 2018 (Signal Separation Evaluation Campaign) – кампания по оценке разделения сигналов 2018 года.

SiSec 2018 MUS – задача, которая решалась во время кампании SiSec 2018. Задача состояла в оценке систем, которые извлекают музыкальные инструменты из популярных музыкальных аудиопотоков, миксов музыкальных инструментов.

MUSDB18 – датасет музыкальных композиций с разделенными стемами, который был составлен в рамках задачи SiSec 2018 MUS.

MSS (music source separation) – задача выделения источников из музыкального потока.

SOTA (state of art) – ”состояние искусства стандарт, признанный в исследовательской среде в рамках какой-либо исследовательской задачи.

MSST-WebUI (Music-Source-Separation-Training WebUI) – веб-интерфейс, который объединяет множество современных моделей разделения источников.

BSS Eval – Blind Source Separation Evaluation.

SDR (Signal to Distortion Ratio) – система управления базами данных.

SIR (source to interference ratio) – объектно-ориентированное программирование.

SAR (sources to artifacts ratio) – .

mir\_eval — это Python-библиотека, которая предоставляет стандартизированный и простой способ оценки систем обработки музыкальной информации (Music Information Retrieval, MIR). Она включает реализацию распространённых метрик для различных задач в области MIR.

STFT (Short-Time Fourier Transform) – кратковременное (оконное) преобразованием Фурье.

# СОДЕРЖАНИЕ

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| <b>ВВЕДЕНИЕ</b> . . . . .                                                                        | 7  |
| <b>1 АНАЛИЗ СОСТОЯНИЯ ИССЛЕДОВАНИЯ В ОБЛАСТИ ИЗВЛЕЧЕНИЯ ИСТОЧНИКОВ</b> . . . . .                 | 8  |
| 1.1 Вклад кампании SiSEC MUS 2018 в задачу разделения источников                                 | 9  |
| 1.2 Способы представления звука для задачи разделения источников                                 | 9  |
| 1.3 Обзор архитектур разделения источников . . . . .                                             | 9  |
| 1.3.1 Классические методы source separation . . . . .                                            | 9  |
| 1.3.2 Современные нейронные архитектуры . . . . .                                                | 9  |
| 1.4 Анализ методов дообучения предобученных моделей разделения источников . . . . .              | 20 |
| 1.4.1 Полное дообучение . . . . .                                                                | 20 |
| 1.4.2 Методы «параметрического» дообучения (адаптация части параметров) . . . . .                | 21 |
| 1.4.3 Адаптация низкого ранга (LoRA, Low Rank Adaptation) .                                      | 22 |
| 1.5 Метрики оценки качества (SDR, SIR, SAR) . . . . .                                            | 24 |
| 1.5.1 Декомпозиция сигнала по методу BSS Eval . . . . .                                          | 25 |
| 1.5.2 Scale-Invariant SDR (SI-SDR) — Масштабно-инвариантное отношение сигнал/искажение . . . . . | 28 |
| 1.6 Выводы . . . . .                                                                             | 28 |
| <b>2 Реализация и проектирование архитектуры программного обеспечения</b> . . . . .              | 29 |
| 2.1 Общая схема взаимодействия компонент разрабатываемой системы . . . . .                       | 29 |
| 2.2 Модуль сбора и подготовки данных. Формирование набора данных                                 | 31 |
| 2.2.1 Загрузка исходных аудиозаписей. Клиент взаимодействия с сервисом Freesound . . . . .       | 33 |
| 2.2.2 Генератор миксов из загруженных аудиофайлов . . . . .                                      | 34 |
| 2.2.3 Состав и количество выборок . . . . .                                                      | 35 |
| 2.2.4 Взаимодействие с HuggingFace . . . . .                                                     | 37 |
| 2.3 Выбор моделей для проведения дообучения и дальнейших экспериментов . . . . .                 | 40 |
| 2.4 Адаптация Open-Unmix для полного дообучения . . . . .                                        | 41 |
| 2.5 Спектральное преобразование для подготовки данных для передачи в Open-Unmix . . . . .        | 41 |
| 2.6 Полное дообучение Open-Unmix . . . . .                                                       | 42 |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 2.6.1    | Полное дообучение Open-Unmix с аугментацией спектрограмм . . . . . | 43        |
| 2.7      | Интеграция LoRA в архитектуру Demucs v4 . . . . .                  | 43        |
| 2.7.1    | Контур дообучения . . . . .                                        | 45        |
| 2.7.2    | Валидационный контур . . . . .                                     | 46        |
| 2.8      | Реализация конвейера обучения, запуска и оценки . . . . .          | 48        |
| 2.9      | Выводы . . . . .                                                   | 50        |
| <b>3</b> | <b>Проведение вычислительных экспериментов . . . . .</b>           | <b>51</b> |
| 3.1      | Протокол проведения экспериментов . . . . .                        | 51        |
| 3.2      | Определение проблемы спектрального перекрытия . . . . .            | 51        |
| 3.3      | Анализ количественных результатов . . . . .                        | 53        |
| 3.4      | Инфраструктура и среда выполнения . . . . .                        | 53        |
| 3.5      | Выводы . . . . .                                                   | 54        |
|          | <b>ЗАКЛЮЧЕНИЕ . . . . .</b>                                        | <b>56</b> |
|          | <b>СПИСОК ЛИТЕРАТУРЫ . . . . .</b>                                 | <b>57</b> |

## **ВВЕДЕНИЕ**

Задача автоматического разделения музыкальных источников является одной из ключевых в области цифровой обработки сигналов и машинного обучения. Технологии выделения отдельных инструментов из полифонических записей находят применение в музыкальном продюсировании, реставрации архивных аудиоматериалов, в системах автоматического анализа музыкальных композиций и интеллектуальной обработки аудиосигналов [1], образовательных платформах и приложениях для караоке.

- Актуальность задачи извлечения инструментов
- Цель работы:
- Задачи исследования

Актуальность (музыкальное ADR, мастеринг, ремиксы, авторские права и т.п.).

Объект, предмет, цель, задачи, научная новизна.

Задача: Адаптация лёгких архитектур для извлечения гитары и пианино из полифонических композиций с учётом спектрального перекрытия (Частотно-временное перекрытие) и последующего восстановления тембрового качества.



# 1 АНАЛИЗ СОСТОЯНИЯ ИССЛЕДОВАНИЯ В ОБЛАСТИ ИЗВЛЕЧЕНИЯ ИСТОЧНИКОВ

Выделение аудио-источников (Audio Source Separation) — процесс разделения заранее определённого набора источников звука из смешанного аудиосигнала. Цель такой задачи — выделить конкретные источники, например вокал, инструменты или фоновый шум, из сложной аудиозаписи (Рисунок 1.1).

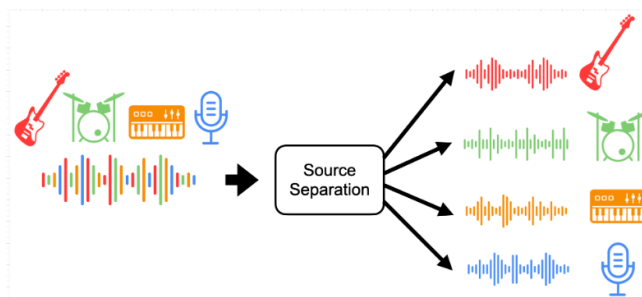


Рисунок 1.1 – Упрощенный процесс разделения источников из аудиозаписи [2]

Для разделения используются алгоритмы обработки сигналов, которые используют различные техники: спектральный анализ, анализ во временной и частотной областях, машинное обучение.

Итогом выполнения задачи декомпозиции смешанного аудиосигнала является выходные данные в виде компонент, представляющих собой изолированные аудиодорожки целевого источника (например, партию гитары, барабанов или вокала) — то есть сами источники. Такая аудиодорожка является единицей музыкального аудиопотока в задаче выделения источников. Когда происходит запись в студии звукозаписи, например, музыкального трека, то происходит запись отдельных музыкальных инструментов и вокала и образуется финальная композиция. И цель задачи извлечения сигналов, в том, чтобы воссоздать эти индивидуальные дорожки из смешанного сигнала-музыкальной дорожки. [3]

Притом, такая задача подразумевает получение из одного аудиотрека (аудиопотока) несколько источников (то есть задача сложнее, чем из потока шума получить какой-то один источник — в статье [3] приводится пример появления задачи — потребность услышать и вычленив из общего шума коктейльной вечеринки голос и рассказ своего собеседника). Тогда можно считать, что исходный музыкальный трек состоит из стемов, и каждый из них можно извлечь из него.

Поэтому необходимо провести обзор методов разделения музыкальных источников, а также методы дообучения нейросетей.

## **1.1 Вклад кампании SiSEC MUS 2018 в задачу разделения источников**

В статье о реализации нейронной сети Open-Unmix [4] говорится про кампанию по оценке разделения сигналов 2018 года (SiSEC 2018 – Signal Separation Evaluation Campaign) [4] [5]. В рамках этой кампании решалась задача SiSEC MUS (оценка систем, которые извлекают музыкальные инструменты из популярных музыкальных аудиопотоков, аудиомиксов (mix) такие стемы были сгруппированы в общие категории :

- барабаны (drums),
- бас (bass),
- вокал (vocals),
- другие музыкальные инструменты и звуки (other).

В результате кампании SiSEC Mus 2018 был выпущен датасет MUSDB18 [5], содержащий 150 музыкальных композиций с разделенными аудио источниками в сумме составляющие 10-часовой плейлист. MUSDB18 и по сей день является самым большим бесплатным датасетом в открытом доступе и активно используется в задачах дообучения нейронных сетей, решающих задачу разделения источников [3] (при необходимости или скудности материалов этого датасета исследователи прибегают к самостоятельному сбору специфичных датасетов [6]).

## **1.2 Способы представления звука для задачи разделения источников**

### **1.3 Обзор архитектур разделения источников**

#### **1.3.1 Классические методы source separation**

ICA

NMF

спектральные методы

#### **1.3.2 Современные нейронные архитектуры**

Современные архитектуры MSS (music source separation) можно условно разделить на два основных класса в зависимости от того, в какой области они обрабатывают аудиосигнал: во временной области (Demucs [3]) или в частотной (Open-Unmix [2] и Spleeter). Далее рассмотрим эти основные виды архитектур и сравним их способы реализации и применение.

Архитектуры, которые представляют собой SOTA (state of art)////////////////////////////////////

**Demucs**

В статье [7] разработчики соц сети Facebook описывают новый подход к методу source separation. Demucs (Hybrid Transformer Demucs) на данный момент является одним из стандартов индустрии (SOTA) задачи извлечения источников.

Demucs [7] – это нейронная сеть глубокого обучения, которая была разработана для решения задачи выделения аудио-источников. В основе нейросети Demucs лежит свёрточная нейронная сеть UNet [8], которая используется для семантической сегментации изображений. То есть Demucs использует подход сети Unet обработки изображений и применяет для обработки волн аудиосигнала по спектрограммам [7].

В статье музыкальная композиция сравнивается с коктейльной вечеринкой, где каждый голос, фоновая мелодия и шум – каждый этот элемент является стемом, и тогда задача выделения голоса собеседника, с которым ведется диалог на вечеринке, является как раз задачей разделения источников. Но задача выделения источников из музыкальной композиции отличается от задачи выделения голоса собеседника на коктейльной вечеринке в первую очередь тем, что в ней интересует в этой задаче выделение не какого-то одного приоритетного источника звука из несвязанного фонового шума, а целый набор тонов и тембров в согласованном потоке.

Авторы утверждают, что на момент постановки проблемы этой статьи основные современные методы извлечения источника оперируют анализом спектрограмм, которые генерируются на основе кратковременного преобразования Фурье (short-time Fourier transform, STFT) [9] [10].

Метод основан на создании маски для magnitude spectrums for each frame and each source, and the output audio is generated by running an inverse STFT on the masked spectrograms reusing the input mixture phase.

Разработчики Demucs основываются на архитектуре Conv-Tasnet и адаптируют ее под извлечение источников (source separation).

Модель Demucs (в актуальных версиях v3/v4 [7] [11]) представляет собой гибридную архитектуру, объединяющую сверточные нейронные сети (CNN) и трансформеры для обработки аудиосигнала во временной области. Основная идея заключается в прямом моделировании звуковой волны, что позволяет избежать артефактов, возникающих при обратном преобразовании Фурье.

Модель Demucs состоит из следующих основных компонент:

- 1) Гибридный энкодер (Hybrid Encoder). Входной сигнал  $x(t)$  преобразуется в скрытое представление (латентные признаки) с помощью одномерных сверточных слоев (1D Convolution). В отличие от чистых временных мо-

делей, Demucs также извлекает признаки из спектрограммы (STFT) того же сигнала. Это позволяет модели использовать преимущества обеих областей: временную точность и частотную структуру.

- 2) Блок разделения (Separator / Transformer). Полученные признаки подаются на вход слоя, состоящего из Transformer-блоков. В отличие от LSTM в Open-Unmix, механизм «самовнимания» (Self-Attention) позволяет модели захватывать глобальный контекст всей композиции, эффективно разделяя источники с длинными зависимостями. Внутри блока применяется маскирование или прямое оценивание источников.
- 3) Гибридный декодер (Hybrid Decoder). Разделенные латентные представления преобразуются обратно в звуковую волну с помощью транспонированных одномерных сверток (Transposed Convolution). Декодер также работает в двух доменах: он восстанавливает временной сигнал и частотное представление, которые затем суммируются.
- 4) Гибридная функция потерь (Hybrid Loss). Обучение оптимизирует ошибку одновременно в двух областях: во временной (L1/L2 loss) и частотной (L1/L2 loss по спектрограмме). Это заставляет модель генерировать сигнал, который звучит естественно (временная область) и имеет правильную спектральную структуру (частотная область).

Модели, работающие во временной области, такие как Demucs, обрабатывают аудиосигнал непосредственно в виде последовательности амплитудных значений (волн). Их главное преимущество заключается в сохранении полной информации о сигнале, включая фазовые характеристики, что критически важно для точного воспроизведения резких атак нот, характерных для многих инструментов, включая гитару и ударные.

Также к преимуществам модели Demucs относятся:

- 1) Высокое качество разделения (SOTA). Demucs демонстрирует одни из лучших результатов на бенчмарках (MUSDB18, FSD50k), превосходя LSTM-подходы (Open-Unmix) по метрикам SDR (Signal-to-Distortion Ratio) и SAR (Signal-to-Artifact Ratio).
- 2) Глобальный контекст (Transformer). Использование механизма самовнимания позволяет учитывать структуру музыкального произведения на больших промежутках времени, эффективно разделяя источники с похожими тембрами.
- 3) Гибридная оптимизация. Совместное обучение на временных и частотных потерях обеспечивает баланс между восприятием сигнала на слух и точностью спектрального разделения.

Но у модели Demucs также есть значительные ограничения, которые создают препятствия по использованию модели в исследовательских работах:

- 1) **Вычислительная сложность.** Архитектура Transformer имеет квадратичную сложность  $O(N^2)$  по отношению к длине последовательности, что требует значительных ресурсов видеопамяти (VRAM) и времени для инференса по сравнению с Open-Unmix.
- 2) **Большое количество параметров.** Demucs v4 содержит сотни миллионов параметров, что затрудняет развертывание модели на мобильных устройствах или встраиваемых системах без дополнительных методов компрессии.
- 3) **Галлюцинации (Artifacts).** В случаях при сильном перекрытии источников трансформер может генерировать несуществующие звуковые артефакты (hallucinations), пытаясь «заполнить» пробелы в спектре.

**Open-Unmix** Существуют модели, работающие в частотной области, такие как Open-Unmix [4] и Spleeter. Такие модели сначала преобразуют аудиосигнал в спектрограмму с помощью быстрого преобразования Фурье (БПФ), где каждый пиксель представляет собой магнитуду (амплитуду) определенной частоты в определенный момент времени. Обработка происходит над этой спектрограммой, после чего полученная маска применяется к исходной спектрограмме для извлечения целевого источника, а затем выполняется обратное преобразование Фурье для получения результирующей аудиодорожки.

Архитектура модели Open-Unmix описана в статье [4] и приведена на рисунке 1.2.

Общая идея модели Open-Unmix заключается в преобразовании исходного файла аудиомикса музыкальных инструментов в формате спектрограммы (на схеме этот блок назван "mix spectrograms") в набор спектрограмм отдельных музыкальных инструментов из этого микса (на схеме это – "target spectrograms").

Модель Open-Unmix состоит из следующих компонентов:

- 1) **Предобработка данных (Cropping 16 kHz)** – так как предобученная модель Open-Unmix обучалась на наборе данных MUSDB18[5], аудиозаписи которого подвергались AAC-сжатию, в результате которого диапазон спектрограмм обрезался до 16 кГц, в модели Open-Unmix на этапе предобработки входного сигнала выполняется «обрезка» спектрограммы по оси частот с исключением частотных ячеек, соответствующих более высоким частотам (больше 16 кГц).
- 2) **Входная нормализация и масштабирование (Input Scaler)** – блок обу-

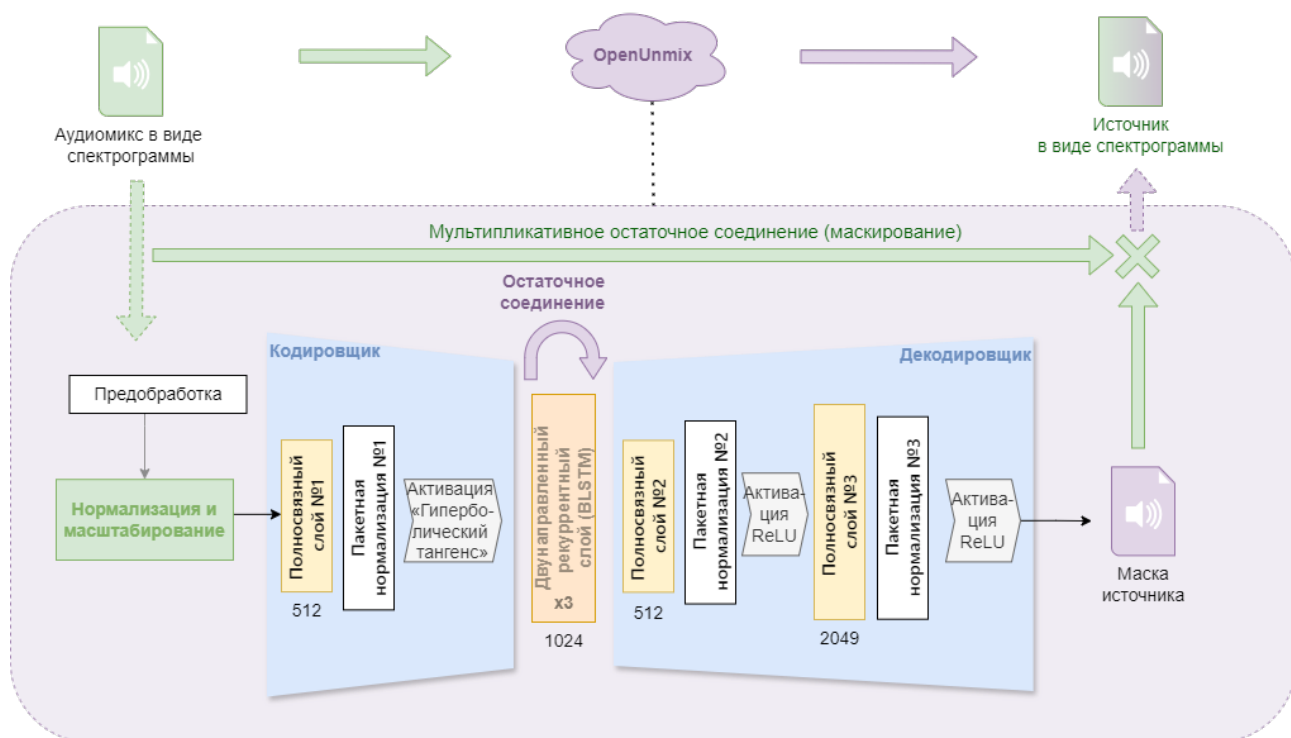


Рисунок 1.2 – Архитектура модели Open-Unmix. Авторский материал

чаемых аффинных параметров. Он выполняет нормализацию входных спектрограмм для улучшения качества работы модели с помощью поэлементного сдвига и масштабирования входных спектрограмм, выравнивая распределение амплитуд по частотным каналам перед подачей на основные слои модели, что ускоряет сходимость градиентного спуска.

3) **Блок кодирования признаков (кодировщик)** – это совокупность механизмов и линейных слоёв внутри единой архитектуры Open-Unmix, которая занимается сжатием входного аудиопотока в компактный набор признаков – то есть выполняет проекцию высокоразмерного частотно-канального представления в компактное скрытое (латентное) пространство. В отличие от классических автоэнкодеров, данный блок не обучается восстанавливать вход, а формирует представление, оптимальное для последующего предсказания маски. В свою очередь модель кодировщика состоит из следующих блоков:

- **Первый полносвязный слой (fc1, Fully Connected)** – линейный слой без смещения, который выполняется сжатие частотно-канального пространства в представление «узкое горлышко» (bottleneck), снижая избыточность данных и формируя компактный вектор признаков.

- **Первый слой пакетной нормализации (bn1, Batch Normalization layer)** – это обучаемый модуль, который выполняет стандартизацию выходных значений полносвязного слоя перед подачей на функцию активации. В архитектуре Open-Unmix данный слой обрабатывает 512-мерное

векторное представление («узкое горлышко»), обеспечивая согласованность признаков перед их подачей на нелинейное преобразование  $\tanh$ .

- **Функция гиперболического тангенса ( $\tanh$ )** – нелинейная функция активации, ограничивающая значения активации диапазонов  $[-1, 1]$ , что необходимо для устойчивой работы рекуррентных слоёв. Формула функции гиперболического тангенса имеет вид:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.1)$$

где  $x$  – входная активация слоя bn1,  $e \approx 2.718$  – основание натурального логарифма.

- 4) **Центральный модуль модели – блок, реализованный на основе двунаправленной долговременной краткосрочной памяти (Bidirectional Long Short-Term Memory, BLSTM).** Данный модуль предназначен для выявления и учёта динамических взаимосвязей между временными отсчётами аудиосигнала, поскольку музыкальная запись характеризуется непрерывным изменением спектрального состава и амплитуды. Архитектурно блок BLSTM представляет собой последовательную цепь из трёх рекуррентных слоёв (обозначено как  $\times 3$  на схеме).

Такая многослойная архитектура обеспечивает формирование признаков по принципу иерархии: начальные слои фиксируют кратковременные локальные особенности звучания (амплитудные огибающие, резкие переходные процессы), тогда как последующие слои последовательно интегрируют эти данные, выделяя более протяжённые музыкальные закономерности (мелодические фрагменты, гармонические последовательности, спектрально-тембровые характеристики).

Для устойчивого обучения глубокой рекуррентной сети и предотвращения потери исходных данных на промежуточных этапах применяется механизм **остаточного соединения (Skip Connection)**.

На схеме он изображён фиолетовой стрелкой над блоком ”Двунаправленный рекуррентный слой”(BLSTM) и обозначает объединение исходного входного тензора с результатом обработки блоком BLSTM. Математически это объединение описано в формуле 1.2:

$$\mathbf{h} = [\mathbf{x} \parallel \mathcal{F}_{\text{BLSTM}}(\mathbf{x})] \quad (1.2)$$

где

- $\mathbf{x}$  – входное представление размера  $H$  (512),
- $\mathcal{F}_{\text{BLSTM}}(\cdot)$  – преобразование, выполняемое стеком BLSTM-слоёв,



- $\parallel$  – операция конкатенации, увеличивающая размерность до  $2H$  (1024).

Данный приём обеспечивает прямой градиентный поток, сохраняет низкочастотную спектральную информацию и ускоряет сходимость оптимизатора без риска деградации представлений при увеличении глубины сети. То есть данные из начала блока копируются в конец, минуя сложные вычисления. Это нужно, чтобы важная информация не потерялась при глубокой обработке.

5) **Блок восстановления размерности (декодировщик)** – набор механизмов и линейных слоёв, отвечающих за восстановление размерности тензора до пространства входной спектрограммы для последующего формирования маски. В отличие от декодера автоэнкодера, данный блок не обучается реконструировать исходный сигнал, а предсказывает коэффициенты маски, которые при поэлементном умножении на входной спектр выделяют целевой источник.

В конфигурации Open-Umix входят следующие компоненты:

- **Второй и третий полносвязные слои (fc2 и fc3)** – слой fc2 проецирует объединённый вектор признаков из расширенного пространства  $2H$  обратно в компактное представление размера  $H$ , а слой fc3 расширяет его до исходного числа частотно-канальных коэффициентов, формируя основу для последующего вычисления весовой маски.

- **Второй и третий слой пакетной нормализации (bn2 и bn3)** – стабилизируют распределение сигналов внутри пакета на промежуточных этапах восстановления. Данная процедура выравнивает масштаб активаций, предотвращает «насыщение» нейронов и делает процесс обновления весов более устойчивым к колебаниям громкости входного аудиосигнала.

- **Функция активации ReLU (Rectified Linear Unit)** – функция применяется после нормализации и на финальном выходе блока. Формально функция определяется как:

$$\text{ReLU}(x) = \begin{cases} 0, & \text{если } x < 0, \\ x, & \text{если } x \geq 0, \end{cases} \quad (1.3)$$

где  $x$  — входная активация.

В отличие от ограниченных  $S$ -образных (сигмоидальных) функций, функция ReLU заменяет все отрицательные значения нулями. Это обеспечивает два важных свойства: во-первых, модель фокусируется только на значимых признаках, подавляя фоновый шум; во-вторых, гарантируется



неотрицательность итоговой маски, что соответствует физической природе амплитудного спектра, поскольку интенсивность частотных компонент не может принимать отрицательные значения.

6) **Мультипликативное остаточное соединение (маскирование) (Multiplicative Skip-Connection)** – механизм финального выделения целевого источника, при котором исходный спектр аудиомикса  $\mathbf{X}$  минует основные слои сети и поэлементно умножается на предсказанную маску  $\hat{\mathbf{M}}$  только на выходе. Такой подход предпочтительнее прямой генерации спектрограммы «с нуля», так как позволяет переиспользовать фазовую информацию из исходного сигнала, избегая артефактов реконструкции. Модель Open-Unmix вычисляет широкополосную маску (full-band mask)  $\hat{\mathbf{M}}$ , значение которой в каждом частотно-временном элементе (бине, bin) отражает долю энергии целевого источника в общем аудиомиксе. Процесс итогового разделения описывается формулой поэлементного умножения:

$$\hat{\mathbf{S}} = \mathbf{X} \odot \hat{\mathbf{M}}, \quad (1.4)$$

где

- $\mathbf{X} \in R^{T \times F}$  — исходная входная спектрограмма аудиомикса (суммарное представление всех инструментов),
- $\hat{\mathbf{M}} \in R^{T \times F}$  — предсказанная нейросетью маска (выход модуля декодирования после функции активации ReLU, обеспечивающей неотрицательность значений),
- $\hat{\mathbf{S}}$  — результирующая спектрограмма целевого источника (например, вокала или гитары).

Таким образом, архитектура Open-Unmix реализует сквозной конвейер частотно-временного разделения источников, в котором входная спектрограмма аудиомикса последовательно проходит через блок кодирования признаков, двунаправленную рекуррентную сеть для моделирования временных зависимостей и блок восстановления размерности, формирующий оценку маски для извлечения целевого сигнала. Финальное выделение изолированной дорожки осуществляется путём поэлементного умножения предсказанной маски на исходную спектрограмму, что обеспечивает восстановление целевого источника с сохранением фазовой информации и минимизацией артефактов, характерных для генеративных моделей.

Модель Open-Unmix имеет как преимущества, так и недостатки среди моделей, решающих задачу разделения источников. Среди преимуществ можно отметить:

- 1) Высокая воспроизводимость и открытость. Авторы модели Open-Unmix называют её эталонной реализацией [4] для задач разделения источников. Код, предобученные веса и документация находятся в открытом доступе, что обеспечивает полную воспроизводимость результатов и упрощает интеграцию в исследовательские и прикладные проекты.
- 2) Эффективное моделирование временного контекста. Применение двунаправленной LSTM-сети (BLSTM) позволяет модели учитывать как предшествующий, так и последующий контекст относительно текущего фрейма [14]. Это преимущество важно для разделения гармонически насыщенных сигналов, где выделение источника часто требует анализа музыкальной фразы целиком.
- 3) Баланс качества и вычислительной сложности. Архитектура демонстрирует конкурентоспособное качество на эталонных наборах данных (например, MUSDB18) при умеренных требованиях к ресурсам по сравнению с более тяжёлыми генеративными моделями (Diffusion, GAN). Этот принцип делает модель пригодной для обработки без требований реального времени и моделирования при отсутствии больших вычислительных мощностей.
- 4) Модульность и расширяемость. Чёткое разделение на блоки кодирования, рекуррентной обработки и декодирования позволяет адаптировать архитектуру под специфические задачи: замену BLSTM на Transformer, добавление многозадачного обучения или расширение на новые классы источников.

Но модель Open-Unmix также имеет следующие ограничения:

- 1) Вычислительная сложность рекуррентных слоёв. Последовательная природа LSTM-ячеек ограничивает возможности параллелизации при обучении и применении. Обработка длинных аудиопоследовательностей требует значительного времени и памяти, что затрудняет применение модели в системах реального времени с низкой задержкой.
- 2) Допущение о независимости источников. Подход на основе идеальных масок (Ideal Ratio Mask) предполагает, что источники в аудиомиксов аддитивны и слабо коррелированы. В случаях сильного спектрального перекрытия (например, вокал и соло-гитара в одном частотном диапазоне) качество разделения может снижаться из-за компромиссов при оценке маски.
- 3) Отсутствие явного моделирования фазы. Поскольку маска применяется только к амплитуде спектрограммы, фазовая информация копируется из

аудиомикса без коррекции. Это может приводить к остаточным артефактам («фазовый шум») в областях, где целевой источник и интерференция имеют близкие частоты, но противоположные фазы.

Архитектура Open-Unmix представляет собой сбалансированное решение для задачи разделения музыкальных источников, сочетающее интерпретируемость, воспроизводимость и конкурентоспособное качество. Основные ограничения модели связаны с вычислительной сложностью рекуррентных блоков и допущениями, лежащими в основе подхода маскирования. Перспективными направлениями развития архитектуры являются: переход от рекуррентных блоков BLSTM к моделям на основе механизма внимания (Transformer), способным анализировать длительные временные зависимости во всём сигнале одновременно; совместное прогнозирование амплитуды и фазы спектра, что позволит повысить чистоту звучания и снизить количество акустических артефактов; а также оптимизация вычислительного конвейера для работы с минимальной задержкой, что сделает модель пригодной для интерактивных сценариев использования (например, в режиме живого исполнения или в составе цифровых аудиоплагинов).

**Spleeter** Spleeter — это система разделения музыкальных источников, разработанная компанией Deezer [15]. Модель основана на архитектуре U-Net [8], работающей в частотной области, и обучена на синтетически сгенерированном датасете из 25 тысяч композиций. В отличие от рекуррентных подходов (Open-Unmix), Spleeter использует полностью сверточную архитектуру типа U-Net, что обеспечивает высокую скорость обработки за счёт параллелизации вычислений.

Spleeter поддерживает разделение на 2, 4 или 5 источников (vocals, drums, bass, piano, other), что делает её одной из первых промышленных систем, ориентированных на широкое применение.

Архитектура модели Spleeter в соответствии с документацией [16] включает следующие компоненты:

- 1) Входное представление (Input Representation). Входной аудиосигнал  $x(t)$  преобразуется в комплексную спектрограмму с помощью кратковременного преобразования Фурье (STFT) с окном 4096 и шагом 1024.
- 2) Энкодер (Encoder). Энкодер извлекает иерархические признаки из входной спектрограммы с помощью последовательности сверточных блоков. Каждый блок включает:

- Свертку  $3 \times 3$  с шагом 2 (downsampling);
- Пакетную нормализацию (Batch Normalization);

- Функцию активации ReLU.

- 3) Блок сжатия (bottleneck – «горлышко бутылки», узкое место) – это центральный блок слоёв, в котором формируется наиболее сжатое представление признаков. Такое сжатие позволяет модели выделять наиболее существенные абстрактные характеристики музыкального сигнала, отфильтровывая шум и несущественные детали.
- 4) Декодер с пропущенными связями. Декодер восстанавливает пространственное разрешение признаков с помощью транспонированных сверток (Transposed Convolution).
- 5) Выходной слой и маскирование. Выход модели представляет собой маску, применяемую к исходной спектрограмме для извлечения целевого источника. Обратное преобразование Фурье (iSTFT) используется для восстановления временного сигнала.

Модель Spleeter имеет следующие преимущества:

- 1) Высокая скорость выполнения запроса. Благодаря полностью сверточной архитектуре без рекуррентных связей, все вычисления могут быть распараллелены на GPU. Обработка трека происходит в 5–10 раз быстрее, чем в реальном времени, что делает модель пригодной для потоковых сервисов.
- 2) Эффективное сохранение деталей. Архитектура модели Spleeter, основанная на модели U-Net с механизмом skip-connections позволяет восстанавливать высокочастотные компоненты, которые часто теряются в архитектурах с сильным сжатием, обеспечивая «прозрачное» звучание.
- 3) Устойчивость к переобучению. Использование пакетной нормализации и умеренной глубины сети (по сравнению с Transformer-моделями) снижает риск переобучения на небольших датасетах.

Также существуют значимые ограничения в работе модели Spleeter:

- 1) Частотные артефакты. Работа исключительно в магнитудной области с сохранением фазы из аудиомикса приводит к артефактам в областях сильного спектрального перекрытия источников.
- 2) Малая гибкость архитектуры. Фиксированная структура U-Net сложнее адаптируется под новые классы источников или многозадачное обучение по сравнению с модульными подходами (Open-Unmix, Demucs).
- 3) Модель Spleeter является устаревшей и не применяется в современных исследованиях.

## 1.4 Анализ методов дообучения предобученных моделей разделения источников

Существуют задачи, в рамках которых средств существующих предобученных нейронных сетей недостаточно и возникает потребность провести над выбранной нейронной сетью ряд действий, позволяющих «научить» её тому контексту, которому она не была «обучена» ранее, в процессе основного обучения, но которого не хватает для решения специфичной исследовательской задачи. При этом чаще всего такое обучение требуется провести без утраты тех «знаний», которым была обучена модель первоначально, так как повторное переобучение модели с нуля – дорогостоящий процесс, требующий больших вычислительных мощностей, больших наборов данных, настройки всех параметров и т.д.

В таких случаях современные исследователи прибегают к подходу дообучения модели, в рамках которого модель не требуется переобучать заново, но необходимо провести её «тонкую настройку» (fine-Tuning) [20] [18] – то есть на уровне механики модели провести некоторые действия, чтобы научить нейронную сеть новым концептам с максимальным сохранением предыдущего накопленного опыта.

В случае задачи разделения источников в аудиосигнале обучение предобученной архитектуры модели с открытым исходным кодом на целевом датасете позволяет адаптировать модель к специфическим классам источников, о которых модель не «знала» и которые не умела извлекать до этого, без необходимости обучения «с нуля».

Метод «тонкой настройки» в настоящее время применяется повсеместно на различных генеративных моделях ИСТОЧНИКИ НУЖНЫ. В зависимости от вычислительных ограничений и объёма данных для решения задачи дообучения применяется одна из трех основных стратегий: полная тонкая настройка, частичная настройка выходного блока и адаптация низкого ранга [19].

### 1.4.1 Полное дообучение

Полное дообучение подразумевает обновление всех весов предобученной модели на новом, целевом наборе данных.

Все параметры предобученной модели размораживаются, и оптимизатор обновляет веса всех слоёв на новом датасете. Обучение проводится с пониженным коэффициентом обучения (learning rate), чтобы не разрушить уже усвоенные общие признаки звука, но позволить сети адаптироваться к новому распределению данных.

В контексте моделей разделения источников звука это означает повторное обучение всей сети на собственном датасете с дорожками тех инструментов, которые модель не умеет извлекать, с возможной корректировкой функции потерь и архитектурных деталей.

Основное преимущество данного подхода — максимальная гибкость адаптации модели к специфике задачи, в том числе к особенностям частотного диапазона и перекрытию гармоник между инструментами. Также к преимуществам можно отнести:

- 1) Простота реализации: не требует модификации архитектуры, чаще всего достаточно внести небольшое количество изменений.
- 2) При достаточном объёме данных демонстрирует наивысшее качество разделения.

В то же время, полное дообучение требует значительных вычислительных ресурсов, длительного времени сходимости, высокого объёма целевого датасета и аккуратного подбора гиперпараметров, поскольку существует риск переобучения и потери уже выученных обобщённых признаков.

#### **1.4.2 Методы «параметрического» дообучения (адаптация части параметров)**

Существует класс методов, которые отталкиваясь от концепции метода полного дообучения, дообучают не полностью всю модель, а только часть её весов: замораживают основную массу весов предобученной модели и обучают лишь малую добавочную часть параметров (обычно  $< 5\%$  от общего числа). Такие методы называют "частичным дообучением" или "параметрически эффективная адаптация" **НУЖНЫ ИСТОЧНИКИ**.

Суть метода заключается в том, что все слои Энкодера и рекуррентного/сверточного блока, размораживается только выходной слой (декодер или классификационная «голова») [20]. Модель использует предобученные признаки общего звукового представления, а перестраивается только механизм маппинга признаков в целевые маски источников [20].

Преимущества такого подхода перед методом полного дообучения заключаются в следующем:

- 1) Минимальные требования к памяти и времени обучения [20].
- 2) Высокая стабильность: модель не теряет общие акустические паттерны [20] [17].
- 3) Простота отладки и быстрая проверка гипотез о составе датасета [20].

Но несмотря на преимущества, у метода есть ограничения:

- 1) Низкая гибкость: если спектры пианино и гитары существенно отличаются от распределения, на котором обучался энкодер, качество разделения будет ограничено «потолком» замороженных признаков [17].
- 2) Не подходит для задач с сильным доменным сдвигом (например, переход от студийных записей к live-концертам или **лоу-фай семплам**) [17].

### 1.4.3 Адаптация низкого ранга (LoRA, Low Rank Adaptation)

Метод LoRA (англ. Low Rank Adaptation – дословно переводится как «адаптация на основе низкого ранга») был разработан для больших языковых генеративных моделей (LLM) [21], но нашёл применение в большом пласте работ по дообучению моделей в машинном обучении ТОЖЕ ИСТОЧНИКИ НУЖНЫ напр SD.

Основное отличие LoRA от методов полного дообучения и частичного дообучения заключается в том, что здесь адаптируется некоторое количество параметров, результатом метода LoRA являются те веса, которые были преобразованы. Низкоранговые методы дообучения основаны на идее замены части весов матриц на низкоранговые приращения, которые обучаемы, а исходные веса при этом остаются неизменными. Подход LoRA схематично представлен на рисунке 1.3.

Суть подхода LoRA представлена в названии метода: исследователи при создании этого метода проверяли гипотезу, которая заключалась в том, что внутренняя размерность (ранг) матрицы весов  $\Delta W = W_{fine\_tuning} - W_0$  (где  $W_{fine\_tuning}$  – веса модели после дообучения,  $W_0$  – веса исходной модели) в больших моделях обладает низким рангом (intrinsic rank), несмотря на то, что исходная матрица весов  $W_0$  имеет полный ранг. Это явление объясняется избыточной параметризацией (overparameterization) современных нейросетей [22]: число обучаемых параметров значительно превышает минимально необходимое для решения задачи, что позволяет оптимизатору концентрировать полезные градиенты в низкоранговом подпространстве без потери обобщающей способности (то есть «предыдущих знаний»).

Такое предположение позволяет декомпозировать изменение весов в виде произведения двух матриц более низкого ранга, чем матрица весов (а не оптимизировать саму матрицу весов изменяемого слоя):

$$\Delta W = BA, \tag{1.5}$$

где

- $A \in R^{r \times k}$  – матрица размерности  $r \times k$ ,
- $B \in R^{d \times r}$  – матрица размерности  $d \times r$ ,



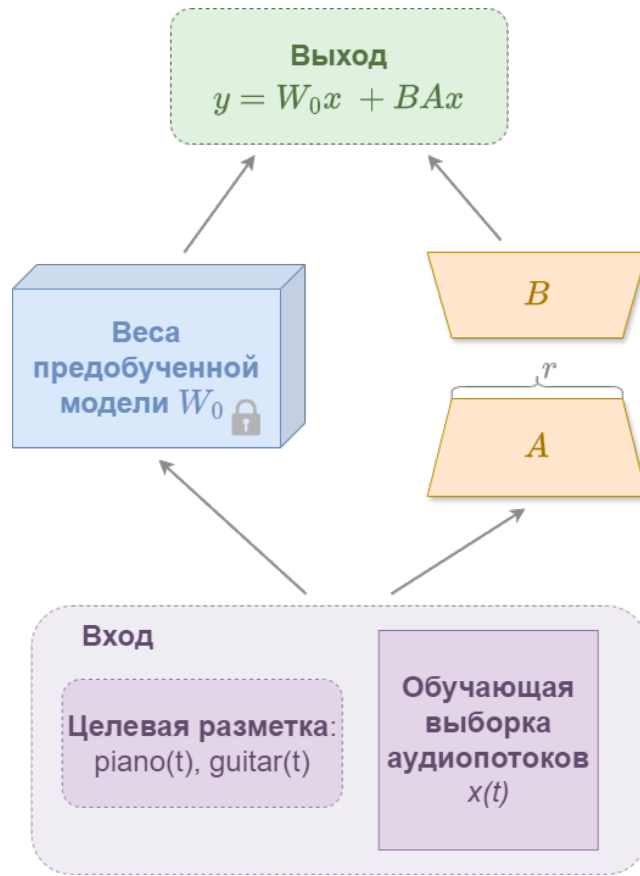


Рисунок 1.3 – Структура подхода LoRa. Собственный материал

- $r \ll \min(d, k)$  – ранг матрицы  $W$ , должен быть как можно меньше.

В процессе дообучения методом LoRA корректируются только матрицы  $A$  и  $B$  (считаются их градиенты и обновляются), что позволяет существенно снизить объём обучаемых параметров и требования к памяти GPU. Модель обучается лишь в низкоранговом подпространстве, которое описывается этими низкоранговыми матрицами, причём веса модели  $W_0$  замораживаются и не преобразуются. Для получения итогового результата матрицы складываются:

$$y = W_0x + \Delta Wx = W_0x + BAx, \quad (1.6)$$

где

- $A$  инициализируется Гауссовским распределением  $A = N(0, \sigma^2)$ ,
- $B$  инициализируется 0,
- $W_0$  – исходная матрица обучаемой модели.

Таким образом, LoRA заменяет полное обновление весов на обучение только компактных адаптеров, что сокращает количество обновляемых параметров на 70–91 % [21], снижает потребление видеопамати и минимизирует риск катастрофического забывания исходных знаний модели, который часто сопутствует методам полного дообучения. В задаче выделения аудио источников, метод LoRA может применяться к отдельным слоям или компонентам модели



(например, слоям свёртки или частотно-временным модулям), что позволяет выполнять адаптацию параметров без изменения архитектуры исходной сети.

Среди преимуществ метода разработчики LoRA приводят следующие [21]:

- 1) Адаптеры LoRA небольшого размера, в связи с чем требования к размеру ресурсов для их расположения достаточно снижены.
- 2) Сниженные требования к видеопамяти (VRAM). Метод не предусматривает выполнение тяжёлых операций расчёта градиентов для большинства параметров модели, следовательно, подход менее ресурсозатратен и поддерживает минимальные требования к железу. Это позволяет проводить эксперименты с крупными архитектурами в средах с ограниченными ресурсами (например, Google Colab в бесплатной версии).
- 3) Модульность и возможность переиспользования. Веса адаптеров A и B хранятся отдельно от базовой модели и могут быть дополнительно загружены или заменены без внесения изменений в архитектуру исходной модели. В рамках задачи извлечения источников, это позволяет поддерживать библиотеку адаптеров для различных инструментов (пианино, гитара, вокал) и комбинировать их при запуске модели.
- 4) Отсутствие катастрофического забывания. Заморозка исходных весов  $W_0$  сохраняет знания, усвоенные моделью на этапе предобучения.

Также можно отметить следующие ограничения метода:

- 1) Необходимость подбора ранга  $r$ . Значение ранга  $r$  является гиперпараметром, требующим эмпирической настройки. Слишком малое  $r$  может ограничивать способность адаптации, а чрезмерно большое  $r$  нивелирует преимущества метода по экономии параметров и памяти при его использовании.
- 2) Ограничения при больших различиях в данных. Гипотеза о низком ранге обновлений  $\Delta W$  может нарушаться при адаптации к задачам, радикально отличающимся от распределения данных предобучения (например, переход от студийных записей к полевым записям с низким соотношением сигнал/шум). В таких случаях полное дообучение может демонстрировать более высокую точность и быть более эффективным решением.

### **1.5 Метрики оценки качества (SDR, SIR, SAR)**

В основных исследованиях, направленных на изучение дообучения моделей разделения источников, полученные результаты оцениваются при помощи метрик, которые являются стандартами в части оценки [23].

Для объективной оценки качества разделения источников для сравнения

соответствующих методов применяются метрики, основанные на сравнении предсказанных сигналов  $\hat{s}_k$  с эталонными (ground truth) источниками  $s_k$ . Данная методология предполагает разложение общей ошибки на две независимые составляющие: влияние компонентов посторонних источников в исходном аудиосигнале (интерференция) и искажения, внесённые алгоритмом разделения (артефакты). Такой подход позволяет независимо оценивать два аспекта качества: эффективность подавления посторонних сигналов (инструментов, шумов) и чистоту извлечения целевого источника (насколько естественно звучит исходный источник).

В данной работе используются метрики семейства BSS Eval (Blind Source Separation Evaluation) [23] и их модификация SI-SDR. Набор оценочных метрик формировался в соответствии с методологией оригинальных исследований сравниваемых архитектур: для Open-Unmix [4] основным показателем выступает SDR, для Demucs [7] применяются SDR, SIR и SAR, а для Spleeter [15] используется полный набор метрик BSS Eval (SDR, SIR, SAR, ISR). Такой подход обеспечивает методологическую согласованность и корректность сравнительного анализа в рамках принятых в предметной области стандартов.

### 1.5.1 Декомпозиция сигнала по методу BSS Eval

Согласно методологии декомпозиции сигнала [23], оценённый источник  $\hat{s}$  представляется в виде суммы ортогональных компонент:

$$\hat{s} = s_{\text{target}} + e_{\text{interf}} + e_{\text{noise}} + e_{\text{artif}}, \quad (1.7)$$

где

- $s$  — эталонный источник,
- $\hat{s}$  — оцениваемый сигнал, полученный в результате работы модели (выходные данные модели),
- $s_{\text{target}}$  — проекция на исходный аудиоисточник,
- $e_{\text{interf}}$  — компонента интерференции (посторонние источники),
- $e_{\text{noise}}$  — компонента шума (фоновый шум, шум оборудования, не связанный с целевыми источниками),
- $e_{\text{artif}}$  — компонента артефактов (искажения алгоритма),

В условиях отсутствия явного шума в датасетах компонента  $e_{\text{noise}}$  часто пренебрегается и объединяется с компонентой  $e_{\text{artif}}$ , так как оба слагаемых представляют собой «нежелательную» часть сигнала, не относящуюся ни к целевому источнику, ни к интерференции. При вычислении метрик SDR, SIR, SAR в популярной библиотеке `mir_eval`, шум  $e_{\text{noise}}$  часто не выделяется отдельно,

если не задан эталонный сигнал шума.

В рамках данного исследования, использующего набор данных без явно-го фонового шума (например, MUSDB18 является такой коллекцией данных), компонента  $e_{\text{noise}}$  считается пренебрежимо малой и объединяется с компонентой артефактов. Таким образом, для расчёта метрик применяется упрощённая декомпозиция:  $\hat{s} \approx s_{\text{target}} + e_{\text{interf}} + e_{\text{artif}}$ .

На основе этой декомпозиции вычисляются три основные метрики (в децибелах, дБ), каждая из которых характеризует определённый аспект качества разделения: SDR отражает общее отношение полезного сигнала к сумме всех ошибок, SIR измеряет эффективность подавления интерферирующих источников, а SAR оценивает уровень артефактов, внесённых алгоритмом обработки.

### **Signal-to-Distortion Ratio (SDR) — Отношение сигнал/искажения**

SDR (Signal-to-Distortion Ratio, отношение сигнала к искажению) является интегральной метрикой, оценивающей общее качество разделения с учётом всех типов ошибок (например, наличие артефактов и наложения на источник других аудиоисточников).

Метрика отвечает на вопрос: «Насколько сильно оценённый сигнал отличается от идеального, учитывая все возможные источники ошибок?». Метрика вычисляет отношение энергии компоненты  $s_{\text{target}}$  к суммарной энергии всех ошибок, возникающих в процессе разделения:

$$\text{SDR} = 10 \log_{10} \left( \frac{\|s_{\text{target}}\|_2}{\|e_{\text{interf}} + e_{\text{artif}}\|_2} \right), \quad (1.8)$$

где  $\|\cdot\|_2$  — Евклидова норма вектора дискретного сигнала.

Значение метрики выражается в децибелах (дБ). Чем выше значение метрики SDR, тем лучше качество полученного источника. Значение 0 дБ означает равенство энергии сигнала и энергии ошибок; положительные значения указывают на преобладание полезного сигнала. Увеличение значения метрики SDR на 10 дБ соответствует десятикратному улучшению отношения сигнал/шум. В исследованиях по разделению музыки [7] [4] [15] метрика SDR служит основным показателем для сравнения моделей.

### **Signal-to-Interference Ratio (SIR) — Отношение сигнал/интерференция**

Метрика SIR измеряет способность модели подавлять другие источники в исходном аудиопотоке. Эта метрика игнорирует артефакты и шум, фокусируясь исключительно на «чистоте» разделения: насколько хорошо алгоритм отделил, например, вокал от аккомпанемента. Высокий SIR означает, что в выделенной дорожке практически не слышно других инструментов. Формула 1.9 описывает математическое определение метрики SIR.

$$\text{SIR} = 10 \log_{10} \left( \frac{\|s_{\text{target}}\|_2^2}{\|e_{\text{interf}}\|_2^2} \right). \quad (1.9)$$

Высокие значения SIR (например, >10–15 дБ) свидетельствуют об эффективном разделении источников. Однако высокий SIR не гарантирует высокого субъективного качества: сигнал может быть «чистым» от других инструментов, но при этом содержать сильные артефакты обработки (эффект «под водой», фазовые искажения), которые метрика SIR не учитывает.

### **Signal-to-Artifacts Ratio (SAR) — Отношение сигнал/артефакты**

Метрика SAR оценивает «чистоту» обработки сигнала самим алгоритмом, игнорируя наличие остатков других источников. Метрика чувствительна к искажениям, которые вносит метод разделения: пре-эхо, фазовые сдвиги, «музыкальный шум», спектральные разрывы. Фактически, значение метрики SAR показывает, насколько естественно звучит выделенный источник, если абстрагироваться от того, что в нём могут присутствовать другие инструменты.

$$\text{SAR} = 10 \log_{10} \left( \frac{\|s_{\text{target}} + e_{\text{interf}}\|_2^2}{\|e_{\text{artif}}\|_2^2} \right). \quad (1.10)$$

Низкие значения метрики SAR часто указывают на наличие слышимых артефактов, даже если значение общей метрики SDR высоко. Для задач, где важно естественное звучание (например, реставрация или улучшение качества), SAR является критически важным показателем. В методах, преобразующих входной аудиосигнал в частотно-временное представление с помощью STFT (Open-Unmix [4], Spleeter [15] и др.), низкое значение метрики SAR может указывать на артефакты, возникающие при некорректном применении маски к спектрограмме.

### **Image-to-Spatial Ratio (ISR) — Отношение «образа» к пространственному искажению**

Метрика ISR измеряет степень искажения тембральных и фазовых характеристик самого целевого источника в процессе разделения. В отличие от метрики SAR, который фиксирует артефакты, добавленные «извне», ISR оценивает, насколько алгоритм изменил форму исходного сигнала. Например, если вокал после разделения стал «глухим» или приобрёл фазовые искажения, это отразится на снижении значения метрики ISR.

$$\text{ISR} = 10 \log_{10} \left( \frac{\|s_{\text{target}}\|_2^2}{\|s_{\text{target}} - s_{\text{filtered}}\|_2^2} \right), \quad (1.11)$$

где  $s_{\text{filtered}}$  — версия целевого сигнала, прошедшая через линейный фильтр, наилучшим образом аппроксимирующий искажения, внесённые алгоритмом.

Высокое значение ISR указывает на то, что в результате выполнения метода, в выделенном аудио сохранился естественный тембр и фазовая структура источника. Метрика ISR особенно релевантна для частотных методов разделения, где применение масок может приводить к нежелательной фильтрации целевого сигнала. В эталонных наборах данных метрика ISR используется для контроля качества сохранения тембральных характеристик. Так, например, в исследованиях архитектуры Spleeter [15] она применяется в составе полного набора BSS Eval для контроля того, что процесс маскирования не вносит нежелательной фильтрации в целевой сигнал.

### 1.5.2 Scale-Invariant SDR (SI-SDR) — Масштабно-инвариантное отношение сигнал/искажение

Классические метрики чувствительны к глобальному масштабу (громкости) сигнала: если модель выдаёт правильный по форме, но более тихий сигнал, значение метрики SDR будет занижено.

Метрика Scale-Invariant SDR (SI-SDR) устраняет эту зависимость, предварительно нормализуя оценённый сигнал по энергии относительно эталона. Метрика оценивает исключительно сходство формы сигнала, игнорируя разницу в громкости.

Пусть  $s$  — эталонный сигнал,  $\hat{s}$  — оцениваемый сигнал. Определим проекцию оценки  $\hat{s}$  на направление эталонного сигнала  $s$ :

$$\alpha = \frac{\langle \hat{s}, s \rangle}{\|s\|_2^2}, \quad s_{\text{proj}} = \alpha s, \quad (1.12)$$

где

- $\langle \cdot, \cdot \rangle$  — скалярное произведение,
- $\alpha$  — оптимальный коэффициент масштабирования.

Тогда ошибка оценивается следующим образом:

$$\text{SI-SDR} = 10 \log_{10} \left( \frac{\|s_{\text{proj}}\|_2^2}{\|s_{\text{proj}} - \hat{s}\|_2^2} \right). \quad (1.13)$$

Оценка SI-SDR особенно полезна для моделей, работающих непосредственно во временной области (waveform-based), таких как Demucs или ConvTasNet [7], где выходная амплитуда может варьироваться в процессе обучения. Кроме того, метрика является дифференцируемой, что позволяет использовать её непосредственно в качестве функции потерь (loss function) при обучении нейронных сетей.

## 1.6 Выводы

## **2 Реализация и проектирование архитектуры программного обеспечения**

В качестве целевой задачи разделения выбрана пара гармонических инструментов — фортепиано и гитара. Данное сочетание является классическим ”сложным случаем” в литературе [27] по музыкальному разделению источников: оба инструмента обладают широкими гармоническими рядами, значительным частотным перекрытием в области средних частот и схожими временными огибающими, что создаёт амплитудную неоднозначность при вычислении масок. Успешное выделение таких источников требует от модели способности распознавать тонкие тембральные и транзиентные паттерны, что напрямую тестирует эффективность стратегии целевого дообучения. Альтернативные пары (например, скрипка и флейта) обладают более выраженным тембральным контрастом и разделяются базовыми моделями с достаточной точностью, что снижает демонстрационный потенциал эксперимента и усложняет статистическое обоснование улучшения метрик.

Предобученная архитектура Open-Unmix реализует принцип одноцелевой сепарации: каждый экземпляр модели оптимизирован под извлечение строго одного источника. Для многоканального разделения официальный пайплайн инстанцирует независимые копии сети, каждая из которых содержит уникальные веса, обученные на соответствующем подмножестве датасета MUSDB18. Обработка выполняется последовательно, а итоговые спектральные маски применяются к исходному представлению смеси с последующим обратным преобразованием Фурье. В рамках данной работы применён аналогичный подход: два независимых экземпляра модели дообучены под извлечение гитары и пианино соответственно.

### **2.1 Общая схема взаимодействия компонент разрабатываемой системы**

Реализованная система дообучения моделей музыкальной сепарации построена по модульному принципу с чётким разделением ответственности между компонентами подготовки данных, адаптации архитектуры, оптимизации и оценки качества. Управление потоком данных и вычислений осуществляется центральным координатором (Trainer), который инициализирует конфигурацию эксперимента, последовательно активирует модули в соответствии с фазой обучения (train/val) и обеспечивает логирование промежуточных результатов. Архитектура системы поддерживает два параллельных пайплайна: для спектральных моделей (Open-Unmix) с внешним STFT-преобразованием и для волновых



моделей (Demucs) с прямой обработкой аудио, что достигается за счёт абстракции интерфейса данных и унифицированного формата тензоров.

Весь пайплайн обучения был разделён на следующие компоненты:

- 1) Модуль сбора и подготовки данных.
- 2) Модуль обучения Open-Unmix.
- 3) Модуль обучения Demucs.
- 4) Среда выполнения обучения.
- 5) Модуль проведения экспериментов и оценки результатов.

Описание взаимодействия между компонентными всей системы представлено в виде компонентной диаграммы на рисунке 2.1.

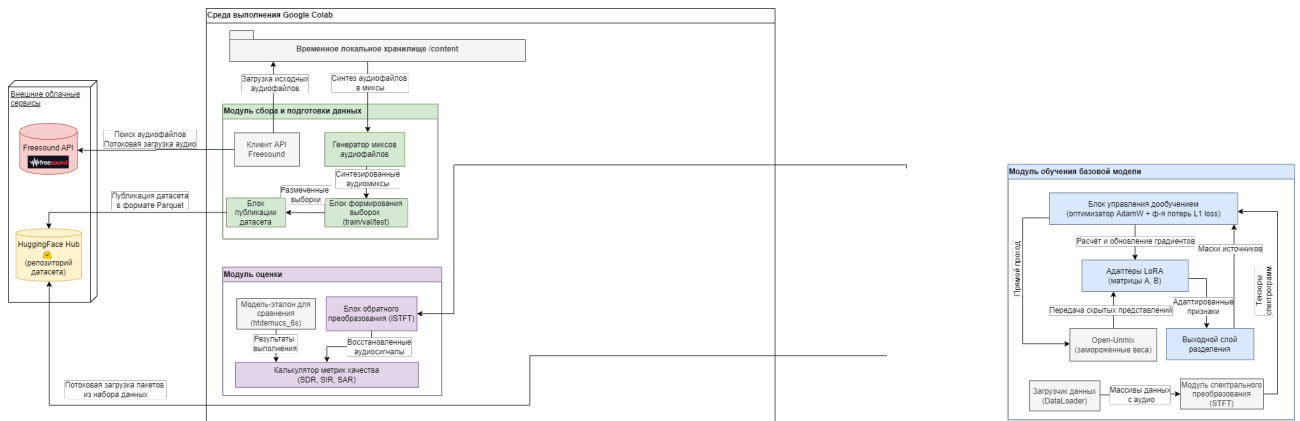


Рисунок 2.1 – Диаграмма взаимодействия между компонентными контейнера дообучения модели Open-Unmix

Диаграмма классов (рисунок 2.2) отражает модульную архитектуру разработанного пайплайна. Подсистема сбора данных (FreesoundDataCollector, AudioMixer) обеспечивает устойчивую загрузку аудио и синтез полифонических миксов с контролируемым соотношением громкости. Для совместимости с различными доменами ввода реализованы два загрузчика: HF\_MSSDataset (спектрограммы  $[2, 2049, T]$  для Open-Unmix) и AudioWrapperDataset (сырые волновые формы  $[2, 176400]$  для Demucs).

Первый оборачивается в SpecAugmentDataset для применения частотно-временных масок на этапе загрузки батча. Блок моделей построен по принципу наследования от абстрактного интерфейса BaseSeparator. Конкретные реализации OpenUnmixAdapter и DemucsAdapter инкапсулируют логику полного обучения и параметрической адаптации (LoRA + FP16 + подвыборка), а BaselineRunner обеспечивает zero-shot инференс предобученных аналогов. Оценка качества унифицирована через UnifiedEvaluator, который автоматически определяет домен модели, вызывает соответствующий декодер (ISTFTDecoder для спектрального случая) и рассчитывает метрики BSS Eval с замером времени инференса. Все артефакты публикуются централизованным оркестратором HuggingFaceManager.

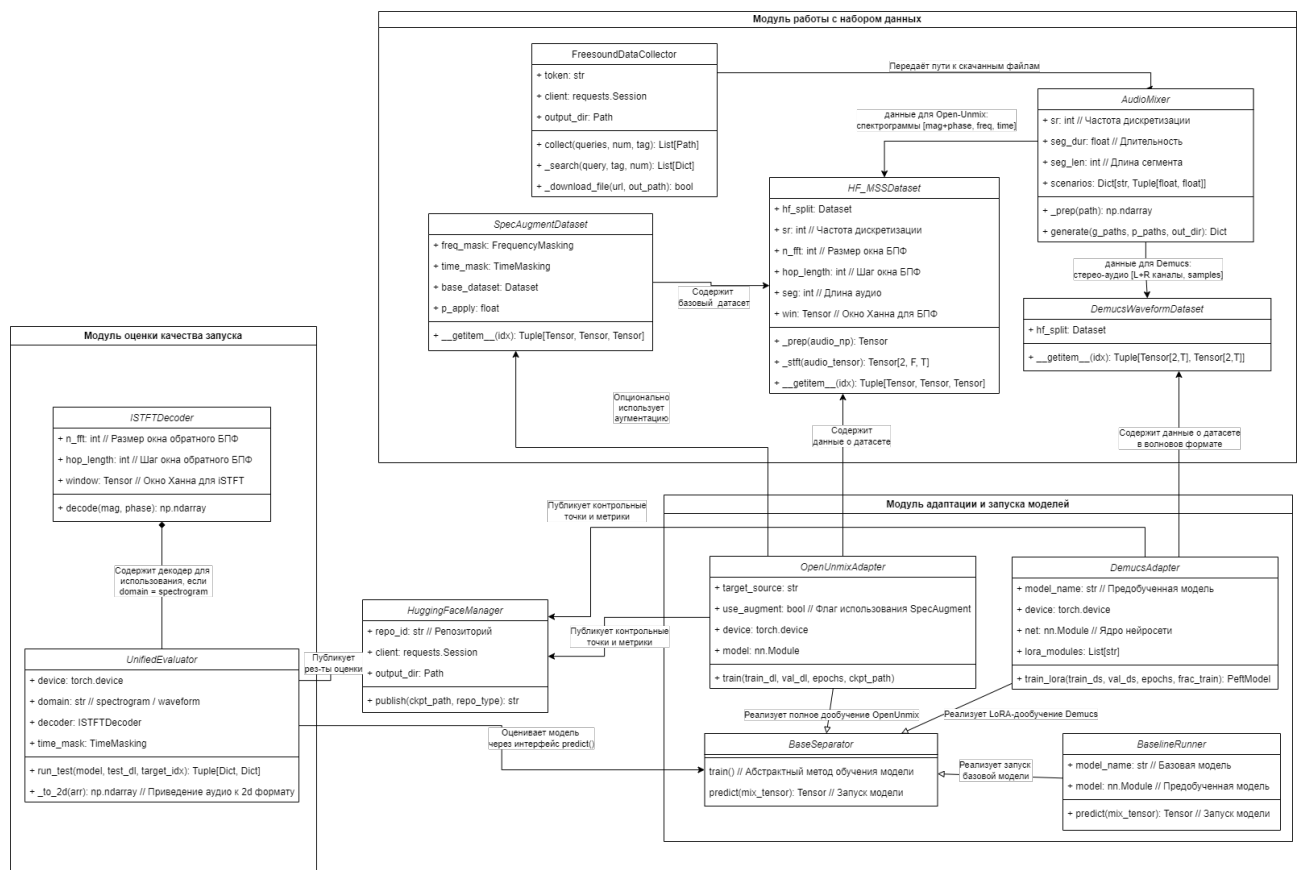


Рисунок 2.2 – Диаграмма классов

## 2.2 Модуль сбора и подготовки данных. Формирование набора данных

Официальные размеченных наборы данных (MUSDB18 [5] или Slakh2100) не предоставляют достаточный набор записей музыкальных инструментов, позволяющих решить задачу разделения пианино и гитары, потому что таких специфичных инструментов нет в наборе, и их смешанных аудио также. Наиболее оптимальным решением было создание собственного набора данных на основе данных из открытых источников. Требовалось: найти отдельные свободно размещённые файлы аудио с отдельно записанным звуком гитары и отдельно пианино, затем с использованием программных средств их смешать между собой в разных комбинациях, чтобы получить разнообразие в датасете, сформировать выборки на два этапа дообучения (обучение и валидация), а также тестовую.

Модуль сбора и подготовки данных представляет собой взаимодействие следующих подмодулей:

- 1) Клиент взаимодействия с сервисом Freesound.
- 2) Генератор миксов из загруженных аудиофайлов.
- 3) Модуль спектрального преобразования (обработчик загруженных аудиофайлов) — это функциональный блок, который отвечает за перевод данных из формата, понятного человеку (звуковая волна), в формат, понят-



ный нейросети (матрица частот). Модуль выполняет преобразование временного сигнала в частотно-временное представление (спектрограмму) методом STFT.

#### 4) Модуль публикации сформированных выборок во внешний сервис HuggingFace

Модуль работы с данными (DataModule) отвечает за загрузку, предобработку и батчирование аудиосэмплов. На этапе инициализации выполняется валидация исходных WAV-файлов, нормализация амплитуды к диапазону  $[-1,1]$  и приведение к фиксированной длине (176 400 отсчётов). Для спектральных моделей активируется подмодуль кратковременного преобразования Фурье (STFT), который преобразует волновой сигнал в комплексную спектрограмму с параметрами  $N\_fft=4096$ ,  $hop=1024$ , окно Ханна, с последующим разделением на магнитуду и фазу. Тренировочный конвейер дополнительно включает модуль спектральной аугментации (SpecAugment), применяющий случайное маскирование по частотной и временной осям с вероятностью 50%. Все выборки (train/val/test) формируются с соблюдением принципа track-wise split и стратификации, что исключает утечку данных и обеспечивает репрезентативность оценки.

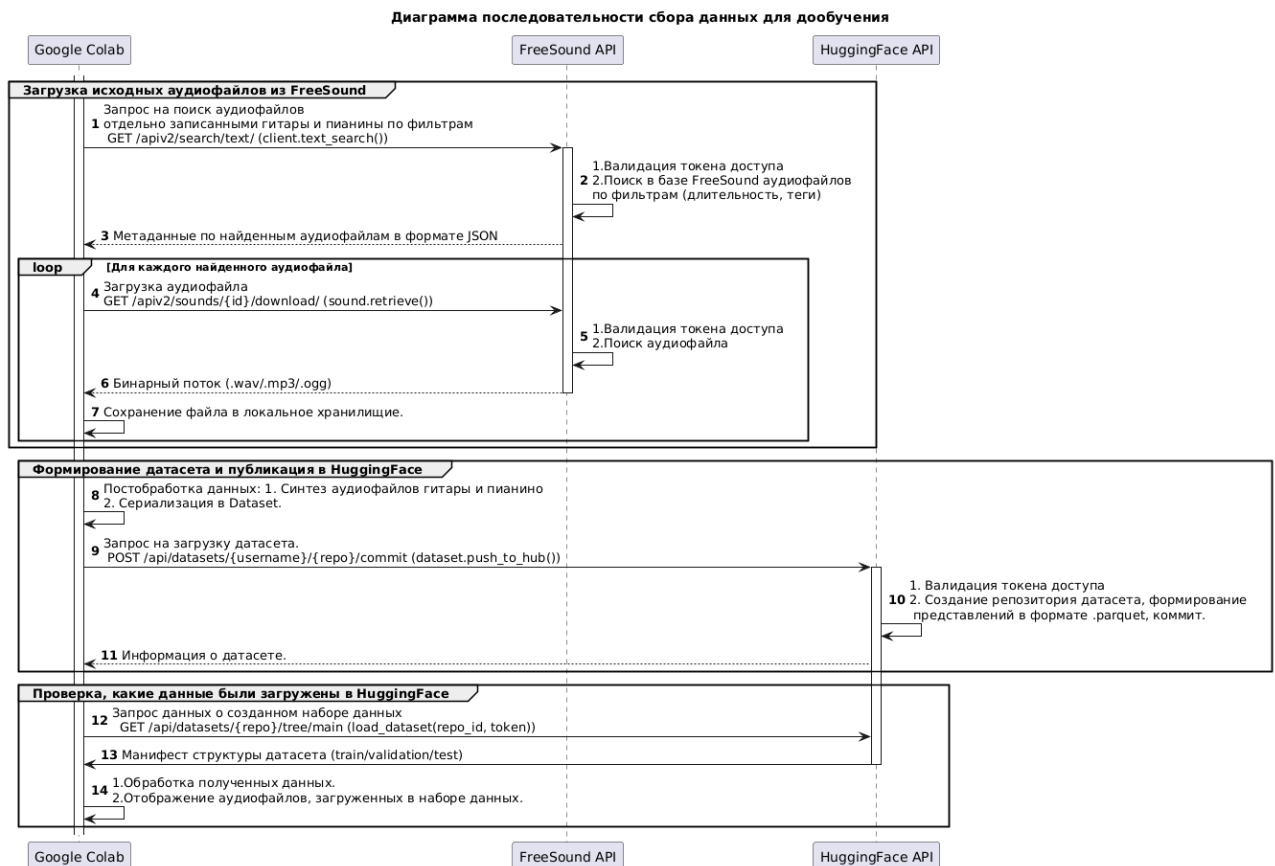


Рисунок 2.3 – Диаграмма последовательности процесса формирования датасета

## 2.2.1 Загрузка исходных аудиозаписей. Клиент взаимодействия с сервисом Freesound

Для формирования тестового набора данных были использованы изолированные аудиозаписи гитары и пианино, загруженные из открытых источников (в частности, через API платформы Freesound — открытого репозитория пользовательских аудиозаписей под свободными лицензиями). Было отобрано по 50 качественных аудиозаписей каждого инструмента, соответствующих требованиям: монофоническая запись, длительность 5–15 секунд, отсутствие сопровождающих инструментов или вокала.

На листинге 2.11 представлена функция загрузки аудиофрагментов гитары и пианино с платформы Freesound.

Листинг 2.1 – Класс поиска и загрузки аудиофрагментов с платформы Freesound

```
1 class FreesoundDataCollector:
2     def __init__(self, api_token: str, output_dir: Path):
3         self.token = api_token.strip()
4         self.output_dir = Path(output_dir)
5         self.session = requests.Session()
6         self.session.headers.update({'Authorization': f'Token {self.token}', 'User-
    ↪ Agent': 'Diploma Project'})
7
8     def _search(self, query, tag, num):
9         url = "https://freesound.org/apiv2/search/text/"
10        params = {'query': query, 'filter': f'tag:"{tag}" duration:[2 TO 10]', 'sort
    ↪ ': 'rating_desc',
11                  'page_size': num, 'token': self.token, 'fields': 'id,name,previews
    ↪ ,license,download'}
12        resp = self.session.get(url, params=params)
13        resp.raise_for_status()
14        return resp.json().get('results', [])
15
16    def _download_file(self, url, out_path):
17        try:
18            resp = self.session.get(url, headers={'Authorization': f'Token {self.
    ↪ token}'},
19                                     params={'token': self.token}, stream=True,
    ↪ timeout=30)
20            if resp.status_code != 200: return False
21            with open(out_path, 'wb') as f:
22                for chunk in resp.iter_content(8192): f.write(chunk)
23            return out_path.stat().st_size > 1024
24        except: return False
25
26    def collect(self, queries, num_per_query, tag="solo", subfolder=None):
27        target = self.output_dir / subfolder if subfolder else self.output_dir
28        target.mkdir(parents=True, exist_ok=True)
29        files = []
30        for q in queries:
```

```

31         for s in self._search(q, tag, num_per_query):
32             safe = "".join(c if c.isalnum() or c in "_-." else "_" for c in s['
↳ name'])
33             ext = s['name'].split('.')[-1].lower()
34             if ext not in ('wav', 'mp3', 'ogg'): ext='wav'
35             p = target / f"{safe}.{ext}"
36             if p.exists() or (s.get('download') and self._download_file(s['
↳ download'], p)):
37                 files.append(p); time.sleep(1.0)
38         return files

```

## 2.2.2 Генератор миксов из загруженных аудиофайлов

На основе загруженных аудиодорожек был синтезирован набор контролируемых миксов путём линейного суммирования сигналов с различными соотношениями громкости (равная громкость, гитара громче, пианино громче), что позволило смоделировать реалистичные сценарии спектрального перекрытия в диапазоне 200–500 Гц — критическом для тембрового различения данных инструментов.

Листинг 2.2 – Класс смешивания аудиофайлов и формирования выборок

```

1 class AudioMixer:
2     def __init__(self, sr=44100, seg_dur=4.0):
3         self.sr, self.seg_dur, self.seg_len = sr, seg_dur, int(sr*seg_dur)
4         self.scenarios = {"balanced":(0.8,0.8), "guitar_loud":(1.0,0.5), "piano_loud
↳ ":(0.5,1.0)}
5     def _prep(self, p):
6         y, _ = librosa.load(str(p), sr=self.sr, duration=self.seg_dur)
7         return np.pad(y, (0, max(0, self.seg_len-len(y))), "constant")[:self.seg_len
↳ ]
8     def generate(self, g_paths, p_paths, out_dir):
9         out_dir = Path(out_dir); out_dir.mkdir(parents=True, exist_ok=True)
10        recs = {"mixture":[], "guitar":[], "piano":[], "scenario":[], "split":[]}
11        for g in g_paths:
12            for p in p_paths:
13                ga, pa = self._prep(g), self._prep(p)
14                for sn,(gg,pg) in self.scenarios.items():
15                    mix = ga*gg + pa*pg
16                    pk = np.max(np.abs(mix)) or 1e-8; mix /= pk
17                    idx = len(recs["mixture"])
18                    pref = f"mix_{idx:04d}_{sn}"
19                    sf.write(str(out_dir/f"{pref}_mixture.wav"), mix, self.sr)
20                    sf.write(str(out_dir/f"{pref}_guitar.wav"), (ga*gg)/pk, self.sr)
21                    sf.write(str(out_dir/f"{pref}_piano.wav"), (pa*pg)/pk, self.sr)
22                    for k,v in {"mixture":str(out_dir/f"{pref}_mixture.wav"),
23                               "guitar":str(out_dir/f"{pref}_guitar.wav"),
24                               "piano":str(out_dir/f"{pref}_piano.wav"),
25                               "scenario":sn}.items(): recs[k].append(v)
26
27        import numpy as np

```

```

28     from sklearn.model_selection import train_test_split
29     idxs = np.arange(len(recs["mixture"]))
30     sc_arr = np.array(recs["scenario"])
31     train_i, temp_i = train_test_split(idxs, test_size=0.3, stratify=sc_arr,
    ↪ random_state=42)
32     val_i, test_i = train_test_split(temp_i, test_size=0.5, stratify=sc_arr[
    ↪ temp_i], random_state=42)
33     sp = ["train"]*len(recs["mixture"])
34     for i in val_i: sp[i]="validation"
35     for i in test_i: sp[i]="test"
36     recs["split"]=sp
37     return recs

```

### 2.2.3 Состав и количество выборок

Для оценки обобщающей способности модели и предотвращения переобучения на ограниченном объёме данных экспериментальный датасет необходимо разделить на обучающую (70% от общего количества смешанных аудиосисточников), валидационную (или отладочную) (15%) и тестовую выборки (15%) [26].

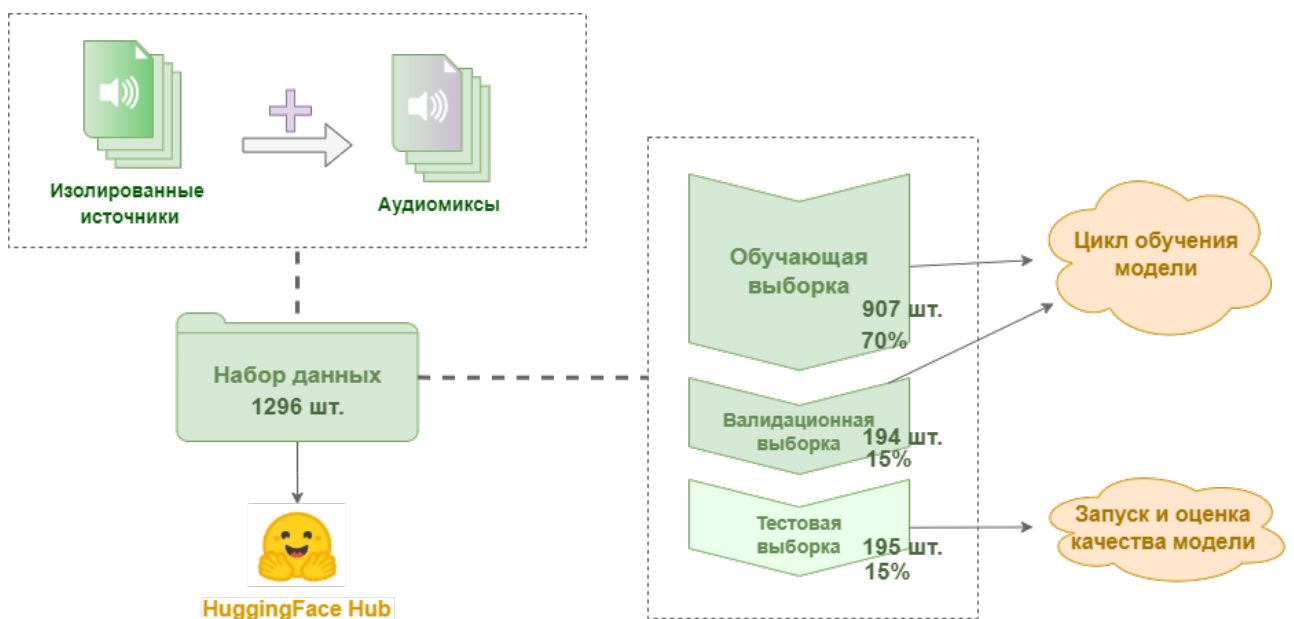


Рисунок 2.4 – Состав выборок

В ходе обучения на каждой эпохе вычислялась ошибка на валидационном наборе без обновления параметров модели. Конфигурация с минимальной валидационной ошибкой сохранялась в формате checkpoint для последующего инференса. Такой подход позволяет объективно оценить способность адаптированной архитектуры к разделению ранее не встречавшихся полифонических композиций.

Экспериментальное исследование проводилось на собственном наборе данных, специально сформированном для задачи выделения акустической ги-

тары из полифонических музыкальных записей. Исходный датасет включает многодорожечные записи, для которых имеются как миксы (смеси всех инструментов), так и отдельные дорожки (stems) целевых источников. Общий объём данных составил 4 тыс. часов аудио, представленного в формате WAV с частотой дискретизации 44.1 кГц и разрядностью 16 бит. Все записи были предварительно нормализованы по громкости и проверены на наличие артефактов записи и наличия других инструментов.

Для обеспечения корректной оценки обобщающей способности моделей исходный датасет был разделён на три непересекающиеся выборки: тренировочную (train), валидационную (validation) и тестовую (test) в пропорции 70%15%15%. Разделение выполнялось на уровне композиций (track-wise split), чтобы избежать утечки данных: все сегменты из одной музыкальной композиции попадали строго в одну выборку. Это гарантирует, что модель не будет оцениваться на данных, которые она уже видела в процессе обучения, даже в виде других временных сегментов той же записи. Для обеспечения репрезентативности каждой выборки применялась стратификация по жанровому признаку и темпу композиции, что позволило сохранить сходное распределение музыкальных характеристик во всех подмножествах данных.

Разделение на обучающую, валидационную и тестовую выборку выполнено с фиксацией случайного *seed* для обеспечения воспроизводимости и сохранения пропорций различных типов миксов (соотношение громкости инструментов – то есть, чтобы в каждый набор было добавлена примерно одинаковая часть аудиофайлов с одним типом/сценарием (сбалансированное смешение, громкая гитара, громкое пианино)), что гарантирует сохранение распределения акустических сценариев во всех подвыборках.

Каждая композиция была нарезана на фиксированные сегменты длительностью 4 секунды (176 400 отсчётов), что соответствует оптимальному компромиссу между вычислительной эффективностью и достаточным временным контекстом для захвата музыкальных фраз. При формировании тренировочной выборки применялась техника спектральной аугментации (SpecAugment): с вероятностью 50% к спектрограммам применялось случайное маскирование по частотной оси (до 32 бинов) и временной оси (до 64 фреймов), что имитирует пропуски спектра и повышает робастность модели. Валидационная и тестовая выборки не подвергались аугментации и представляли собой «чистые» данные для объективной оценки качества. Итоговая тестовая выборка содержала 195 независимых сэмплов, на которых проводилась финальная оценка всех метрик качества (SDR, SIR, SAR, SI-SDR).

Несмотря на относительно небольшой объём тестовой выборки (195 сэмплов), её размер является статистически достаточным для надёжной оценки качества разделения источников. Расчёт показал, что при ожидаемом стандартном отклонении метрики SDR порядка 0.5 дБ и доверительной вероятности 95%, погрешность оценки среднего составляет менее 0.1 дБ, что значительно меньше наблюдаемых различий между моделями (2.5–4.0 дБ). Кроме того, стратифицированное разделение обеспечило покрытие основных музыкальных жанров и темпов, представленных в датасете, что позволяет считать результаты исследования репрезентативными для задачи выделения акустической гитары из полифонических записей.

На листинге 2.3 представлен скрипт разделения всех смешенных аудиоисточников на три основные выборки для применения далее на всех шагах дообучения и тестирования работы модели.

Листинг 2.3 – Скрипт разделения всех смешенных аудиоисточников на три основные выборки

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3
4 # Фиксируем seed детерминированность()
5 np.random.seed(42)
6
7 # Сначала отделяем тест (15%)
8 train_val, test = train_test_split(all_indices, test_size=0.15, random_state=42,
9     ↪ stratify=scenario_labels)
10
11 # Делим оставшееся на train/val (80/20 от остатка → ~70/15 от общего)
12 train, val = train_test_split(train_val, test_size=0.176, random_state=42, stratify=
13     ↪ scenario_labels[train_val])
```

## 2.2.4 Взаимодействие с HuggingFace

### Публикация коллекции данных в HuggingFace

Публикация экспериментального датасета в HuggingFace Hub реализуется через последовательный конвейер обработки. Модуль `FreesoundDataCollector` обеспечивает аутентифицированную загрузку исходных аудиозаписей, которые передаются в `AudioMixer`. Миксер синтезирует полифонические примеры, применяет фиксированные сценарии соотношения громкости, сохраняет файлы в локальное хранилище и формирует структурированный словарь метаданных, содержащий пути к файлам и стратифицированную разметку по сплитам (`train`, `validation`, `test`). Данный словарь преобразуется в объекты библиотеки `datasets` и публикуется в HuggingFace Hub через метод `push_to_hub()`, инкапсулированный в `HuggingFaceManager`.

Загрузчики HF\_MSSDataset и DemucsWaveformDataset в процессе публикации не участвуют; они активируются исключительно на этапе обучения для динамической конвертации загруженных с Hub файлов в тензорные представления, требуемые архитектурами Open-Unmix и Demucs.

Публикация датасета реализуется через слабосвязанный конвейер обработки данных. Модуль AudioMixer не передаёт свои объекты в другие компоненты; он формирует и возвращает стандартный Python-словарь (Dict[str, List]), содержащий пути к сгенерированным .wav-файлам и стратифицированную разметку по обучающей, валидационной и тестовой выборкам. Данный словарь выступает контрактом данных и передаётся в HuggingFaceManager, который конвертирует его в объекты библиотеки datasets, применяет типизацию Audio() и отправляет в облачный репозиторий методом push\_to\_hub(). Отсутствие прямой типовой зависимости между модулями генерации и публикации обеспечивает гибкость пайплайна: при необходимости заменить AudioMixer на другой генератор достаточно лишь соблюсти структуру возвращаемого словаря.

Листинг 2.4 – Класс смешивания аудиофайлов и формирования выборок

```
1 class HuggingFaceManager:
2     def __init__(self, repo, token=None):
3         self.repo = repo
4         if token: huggingface_hub.login(token=token)
5         else: huggingface_hub.notebook_login()
6     def publish(self, ckpt, metrics, desc, rtype="model"):
7         huggingface_hub.create_repo(self.repo, rtype, exist_ok=True)
8         huggingface_hub.upload_file(ckpt, Path(ckpt).name, self.repo)
9         huggingface_hub.upload_file(metrics, "metrics.json", self.repo)
10        rd = Path("/tmp/README.md")
11        rd.write_text(f"---\nlicense: mit\ntags:\n- audio-separation\n- pytorch\n
    ↪ ---\n# {self.repo}\n\n{desc}")
12        huggingface_hub.upload_file(rd, "README.md", self.repo)
13        print(f"📄 https://huggingface.co/{self.repo}")
```

**Загрузка данных из HuggingFace для работы с моделями в среде выполнения** Для повышения устойчивости модели к спектральным искажениям применён паттерн Decorator: базовый загрузчик HF\_MSSDataset обернут в класс SpecAugmentDataset, который на этапе загрузки каждого батча с вероятностью 50% маскирует частотную и временную оси. Такой подход (on-the-fly augmentation) не требует предварительной генерации аугментированных версий на диске, экономит память и обеспечивает бесконечное разнообразие обучающих примеров за счёт стохастической природы маскирования.

Модуль системы отвечает за сохранение артефактов обучения и обес-



печение воспроизводимости результатов. Обученные адаптеры или полные веса моделей сохраняются в формате `safetensors`, совместимом с экосистемой `Hugging Face`. Конфигурация эксперимента (гиперпараметры, версия кода, `seed`) сериализуется в JSON-файл и прикрепляется к чекпоинту. Модуль предоставляет утилиты для загрузки адаптированной модели в сторонних проектах и автоматической публикации артефактов на `Hugging Face Hub`. Все компоненты системы поддерживают детерминированный режим (фиксация `random seed` для Python, NumPy, PyTorch), что гарантирует воспроизводимость результатов при повторном запуске эксперимента.

Листинг 2.5 – Класс смешивания аудиофайлов и формирования выборок

```

1 class HF_MSSDataset(Dataset):
2     def __init__(self, hf_split, n_fft=4096, hop_length=1024, segment_sec=4.0):
3         self.hf, self.sr = hf_split, 44100
4         self.n_fft, self.hop = n_fft, hop_length
5         self.seg = int(self.sr * segment_sec)
6         self.win = torch.hann_window(self.n_fft)
7     def __len__(self): return len(self.hf)
8     def _prep(self, x):
9         t = torch.from_numpy(x).float().unsqueeze(0)
10        return torch.nn.functional.pad(t, (0, self.seg-t.shape[1]))[:, :self.seg]
11    def _stft(self, t):
12        s = torch.stft(t, self.n_fft, self.hop, self.n_fft, self.win, True,
13        ↪ return_complex=True).squeeze(0)
14        return torch.stack([s.abs(), s.angle()], dim=0)
15    def __getitem__(self, i):
16        it = self.hf[i]
17        return self._stft(self._prep(it["mixture"]["array"])), \
18                self._stft(self._prep(it["guitar_source"]["array"])), \
19                self._stft(self._prep(it["piano_source"]["array"]))
20
21 class AudioWrapperDataset(Dataset):
22     def __init__(self, hf_split):
23         self.hf = hf_split
24     def __len__(self):
25         return len(self.hf)
26     def __getitem__(self, i):
27         it = self.hf[i]
28         def to_st(x):
29             x = torch.tensor(x).float()
30             ↪ return x.unsqueeze(0).repeat(2,1)[: , :4*44100] if x.dim()==1 else x[: ,
31             ↪ :4*44100]
32         return to_st(it["mixture"]["array"]), to_st(it["guitar_source"]["array"])

```



## 2.3 Выбор моделей для проведения дообучения и дальнейших экспериментов

Demucs требует 8–12 ГБ VRAM и Transformer-оптимизацию, что превышает лимиты Colab Free и усложняет fine-tuning на кастомных классах. Open-Unmix даёт воспроизводимый baseline с возможностью адаптации выходного слоя под пианино/гитару при 4 ГБ VRAM, что соответствует условиям эксперимента

выбор между Spleeter и Open-Unmix определяется целевым применением. Spleeter предпочтительна для задач, требующих высокой пропускной способности и низкой задержки (онлайн-сервисы, мобильные приложения), где допустим компромисс в качестве ради скорости. Open-Unmix, в свою очередь, является оптимальным выбором для исследовательских задач и приложений, где критично максимальное качество разделения и возможность тонкой настройки под специфические классы источников. Обе модели представляют собой частотные подходы с маскированием, но различаются фундаментально в способе моделирования зависимостей: пространственная иерархия (CNN) против временной рекуррентности (BLSTM).

В ходе сравнительного анализа существующих подходов к задаче разделения музыкальных источников в качестве базовой модели была выбрана архитектура Open-Unmix. Данное решение обусловлено совокупностью технических, вычислительных и методологических факторов, напрямую соответствующих целям настоящего исследования.

Во-первых, Open-Unmix позиционируется авторами как reference implementation (эталонная реализация) [4], что обеспечивает высокую прозрачность кода, модульность архитектуры и стабильность процесса дообучения. В отличие от трансформерных моделей (например, Demucs v4), требующих значительных объёмов видеопамяти и длительного времени сходимости [7], Open-Unmix использует двунаправленные LSTM-блоки с пакетной нормализацией. Такая конфигурация существенно снижает вычислительную нагрузку и позволяет проводить эксперименты в среде Google Colab без риска исчерпания квот GPU или нестабильности обучения на бесплатных тарифах и мощностях.

Во-вторых, частотно-временной подход с мультипликативным маскированием ( $\hat{S} = X \odot \hat{M}$ ) демонстрирует высокую эффективность при работе с гармоническими инструментами, такими как фортепиано и акустическая/электргитара [17]. Спектральные огибающие данных инструментов обладают выраженной структурой в STFT-представлении, что позво-

ляет модели обучаться устойчивым маскам присутствия даже при частичном частотном перекрытии. Архитектура поддерживает transfer learning: предобученные веса для стандартных классов (вокал, бас, ударные) могут быть адаптированы под целевые инструменты без полного перезапуска оптимизации, что критически важно при ограниченном объёме размеченных данных [4].

В-третьих, в сравнении с альтернативными решениями, Open-Unmix обладает архитектурной гибкостью, необходимой для кастомизации под задачу. Модель Spleeter использует softmax-нормализацию масок по каналам источников, что жёстко фиксирует их количество и затрудняет добавление новых классов без перестройки выходного слоя. Demucs, работая в гибридном временно-частотном домене, генерирует источники напрямую, что усложняет интерпретацию промежуточных представлений и требует значительных ресурсов для тонкой настройки [7]. Open-Unmix же предсказывает независимые маски для каждого инструмента, сохраняя аддитивность аудиомикса и позволяя масштабировать выходной слой под произвольное число целевых классов [4].

Таким образом, выбор Open-Unmix представляет собой сбалансированное решение, сочетающее воспроизводимость, вычислительную доступность и архитектурную гибкость. Модель полностью соответствует требованиям исследования по извлечению партий фортепиано и гитары, обеспечивая возможность дообучения в условиях ограниченных вычислительных ресурсов при сохранении академической строгости и сопоставимости результатов с современными бенчмарками.

## 2.4 Адаптация Open-Unmix для полного дообучения

## 2.5 Спектральное преобразование для подготовки данных для передачи в Open-Unmix

В разработанном пайплайне предварительной обработки данных для модели Open-Unmix применяется алгоритм кратковременного преобразования Фурье (Short-Time Fourier Transform, STFT) с параметрами, оптимизированными для задачи музыкальной сепарации. Исходный аудиосигнал  $x[t]$  дискретизируется с частотой 44.1 кГц и разбивается на перекрывающиеся сегменты длиной  $N = 4096$  отсчётов (93 мс) с шагом  $H = 1024$  (23 мс). Каждый сегмент умножается на оконную функцию Ханна  $w[n]$  для минимизации спектральных искажений на границах, после чего к нему применяется быстрое преобразование Фу-

рье (БПФ):

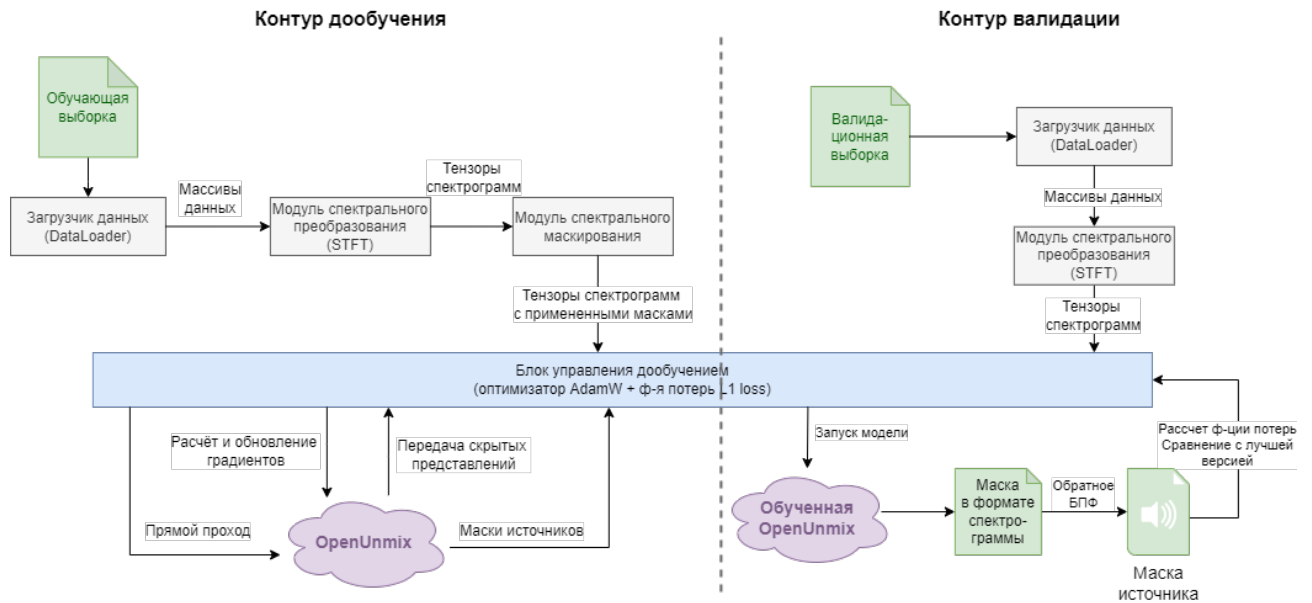


Рисунок 2.5 – Схема системы дообучения

## 2.6 Полное дообучение Open-Unmix

Листинг 2.6 – Класс смешивания аудиофайлов и формирования выборок

```

1 class BaseSeparator:
2     def train(self, *a, **kw): raise NotImplementedError
3     def predict(self, x): raise NotImplementedError
4
5 class OpenUnmixAdapter(BaseSeparator):
6     def __init__(self, target='guitar', aug=False):
7         self.target, self.aug = target, aug
8         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
9         self.model = torch.hub.load('sigsep/open-unmix-pytorch', 'umxhq').
10         target_models['vocals'].to(self.device)
11         self.model.train()
12         for p in self.model.parameters(): p.requires_grad=True
13     def train(self, tr_dl, vl_dl, ep=20, ckpt='/content/unmix_ckpt.pt'):
14         opt = optim.AdamW(self.model.parameters(), lr=1e-4, wd=1e-3)
15         sched = optim.lr_scheduler.CosineAnnealingLR(opt, ep, 1e-5)
16         loss_fn = nn.L1Loss()
17         for e in range(1, ep+1):
18             self.model.train()
19             for m,t,_ in tqdm(tr_dl, leave=False):
20                 m,t = m.to(self.device), t.to(self.device)
21                 opt.zero_grad(); loss = loss_fn(self.model(m), t)
22                 loss.backward(); torch.nn.utils.clip_grad_norm_(self.model.
23                 parameters(),1.0); opt.step()
24                 self.model.eval()
25                 v = sum(loss_fn(self.model(m.to(self.device)), t.to(self.device)).item()
26                 for m,t,_ in vl_dl)/len(vl_dl)
27                 print(f"Ep {e} | Val: {v:.4f}")
28                 torch.save({'epoch':e, 'state':self.model.state_dict(), 'val':v}, ckpt)

```

```

26         sched.step()
27     def predict(self, x): return self.model(x)

```

### 2.6.1 Полное дообучение Open-Unmix с аугментацией спектрограмм

```

1 import torch, random, torchaudio
2 from torch.utils.data import Dataset, Subset, DataLoader
3
4 class SpecAugmentDataset(Dataset):
5     def __init__(self, base_ds, p=0.5, fm=32, tm=64):
6         self.base, self.p = base_ds, p
7         self.fm, self.tm = torchaudio.transforms.FrequencyMasking(fm), torchaudio.
8         ↪ transforms.TimeMasking(tm)
9     def __len__(self): return len(self.base)
10    def __getitem__(self, i):
11        m,g,p = self.base[i]
12        if random.random() < self.p:
13            m[0] = self.tm(self.fm(m[0].unsqueeze(0))).squeeze(0)
14        return m,g,p

```

В ходе сравнительного анализа существующих подходов к задаче разделения музыкальных источников в качестве базовой модели была выбрана архитектура Open-Unmix. Данное решение обусловлено совокупностью технических, вычислительных и методологических факторов

Результаты по переобученной Open-Unmix (умеет извлекать только гитару/пианино): Независимое дообучение модели Open-Unmix под извлечение акустической гитары на кастомном датасете показало высокие метрики качества сепарации: SDR достигла 10.7 dB, SIR – 17.24 dB, что свидетельствует об эффективном подавлении интерференции пианино и сохранении тембральных характеристик целевого инструмента. Метрика SAR приняла значение inf, что в рамках стандарта BSS Eval указывает на отсутствие измеримых спектральных артефактов в тестовых сегментах (энергия ошибок < 1e-12). Для корректного отображения в отчётах значение SAR было ограничено верхней границей шкалы (40.0 dB). Общее время инференса на тестовой выборке из 195 сэмплов составило 43.54 сек (□4.48□ сэмплов/сек на CPU), что подтверждает вычислительную эффективность архитектуры для последующего развёртывания в ресурсоограниченных средах.

## 2.7 Интеграция LoRA в архитектуру Demucs v4

Дообучение модели htdemucs v4 выполнено методом параметрически эффективной адаптации LoRA (Low-Rank Adaptation). В отличие от классического полного дообучения (full fine-tuning), базовые веса всех свёрточ-

ных, трансформерных и линейных слоёв остаются замороженными (requires\_grad\_...  
**В каждый полносвязный слой (nn.Linear) внедряются низкомерные адаптерные матрицы  $A \in \mathbb{R}^{d \times r}$  и  $B \in \mathbb{R}^{r \times d}$  с рангом  $r = 8$ , что добавляет лишь 491 520 обучаемых параметров (0.42% от исходных 115 миллионов параметров). Прямой проход вычисляется как  $h = W \cdot x + B \cdot A \cdot x$ , где  $\alpha=16$  масштабирует вклад адаптеров, а dropout=0.05 предотвращает ко-адаптацию признаков. Поскольку htdemucs изначально обучена на 4 источника [Vocals, Drums, Bass, Other], в работе применён transfer learning без модификации выходного головы: первый выходной канал (индекс 0) перепрофилирован под целевой инструмент (гитара). Функция потерь вычисляется только для этого канала через L1Loss, что исключает необходимость расширения размерностей или добавления новых слоёв. Обучение проводилось на 30**

**Анализ совместимых слоёв: выявление линейных и одномерных свёрточных проекций в Hybrid Transformer, где применим низкоранговый метод. Реализация адаптеров: выбор ранга  $r \in [8, 16]$ , настройка коэффициента  $\alpha$ , заморозка базовых весов, передача параметров адаптеров в оптимизатор. Контроль потребления памяти: применение gradient\_checkpointing, квантование базовых весов (опционально), батч-стратегии. Рекомендуемые элементы: Блок-схема внедрения LoRA-модулей; таблица распределения обучаемых параметров по слоям.**

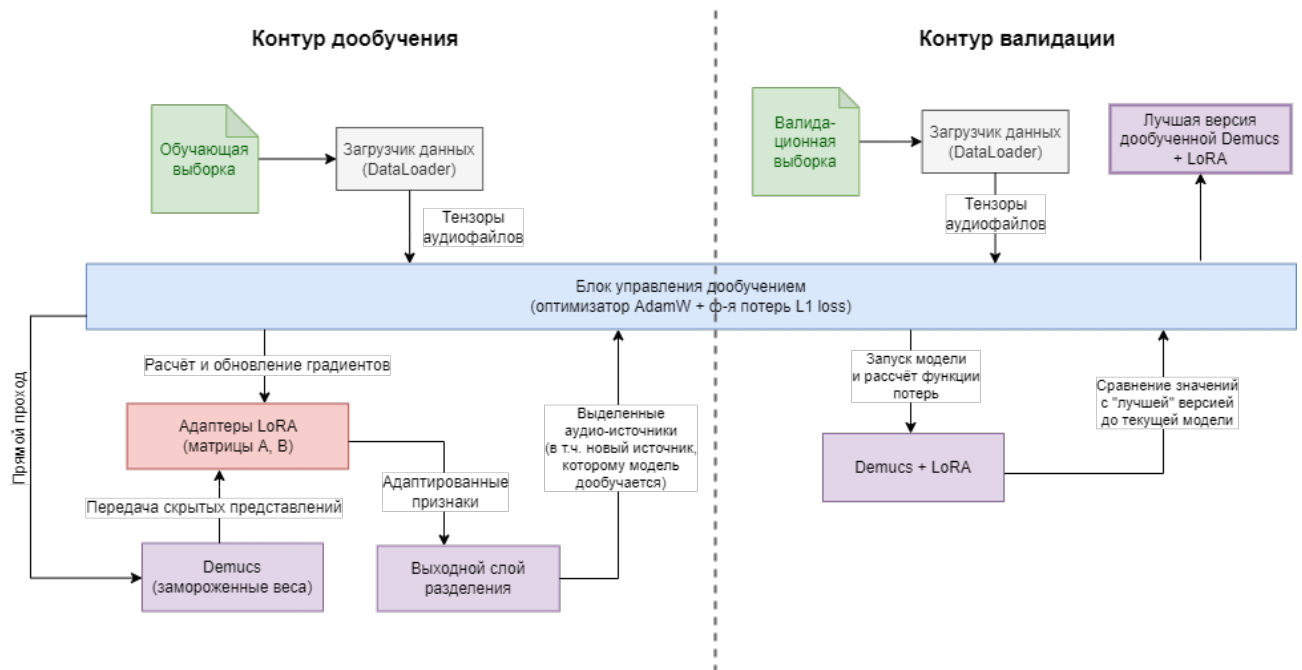


Рисунок 2.6 – Контур дообучения модели Demucs

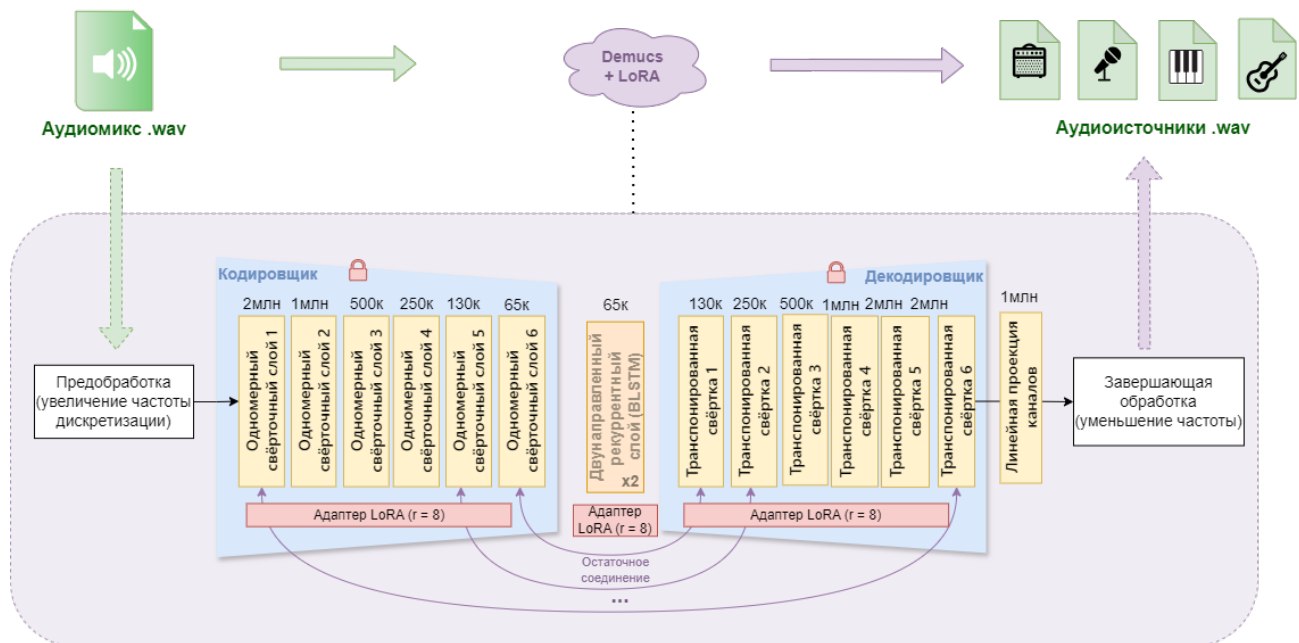


Рисунок 2.7 – Архитектурные изменения в Demucs

### 2.7.1 Контур дообучения

Реализованная система дообучения построена по модульному принципу с чётким разделением ответственности между компонентами обработки данных, архитектурой нейронной сети и оптимизационным контуром. Поток данных начинается с модуля загрузки и предобработки (AudioWrapper), который считывает исходные WAV-файлы, выполняет нормализацию амплитуды, приведение к фиксированной длине (176 400 отсчётов) и формирование батчей. Для спектральных моделей (Open-Unmix) далее активируется модуль кратковременного преобразования Фурье (STFT), преобразующий волновой сигнал в комплексную спектрограмму с последующим разделением на магнитудную и фазовую компоненты. Полученные тензоры формы  $[B, 2, F, T]$  подаются на вход ядра модели, в то время как для волновых архитектур (Demucs) данные передаются напрямую, минуя внешнее спектральное преобразование.

Центральным элементом системы является модуль нейросетевой архитектуры, поддерживающий два режима работы: полное дообучение (Full Fine-Tuning) и параметрически эффективная адаптация (LoRA). В режиме Full Fine-Tuning все параметры модели размораживаются и участвуют в обновлении весов. В режиме LoRA базовые веса предобученной модели фиксируются (`requires_grad=False`), а в целевые полносвязные слои механизма самовнимания и feed-forward блоков внедряются низко-ранговые адаптерные матрицы. Прямой проход через модель вычисляет выход как сумму базового преобразования и адаптерной поправки. Выходной слой

формирует либо спектральную маску (для Open-Unmix), которая применяется к амплитуде смеси с последующим iSTFT, либо непосредственно волновой сигнал целевого источника (для Demucs).

Модуль управления обучением координирует вычисление функции потерь, обратное распространение ошибки и обновление параметров. Функция потерь L1 loss рассчитывается во временной области как среднее абсолютное отклонение между предсказанным и эталонным аудиосигналом целевого источника. Градиенты от функции потерь передаются оптимизатору AdamW, который обновляет обучаемые параметры (все веса при Full FT или только матрицы A и B при LoRA) с применением регуляризации веса (weight decay) и планировщика скорости обучения. Параллельно работает изолированный валидационный контур: модель переводится в режим eval(), аугментация отключается, вычисления выполняются в контексте torch.no\_grad(). Среднее значение val\_loss сравнивается с сохранённым минимумом, и при достижении нового рекорда веса адаптеров экспортируются в чекпоинт, что обеспечивает отбор состояния с наилучшей обобщающей способностью.

Все модули системы объединены единым интерфейсом управления через конфигурационный файл, что позволяет гибко переключаться между архитектурами, методами адаптации и гиперпараметрами без изменения кода. Логирование процесса обучения (значения потерь, метрики, время эпохи) осуществляется через модуль мониторинга, совместимый с TensorBoard. Финальные артефакты — обученные адаптеры или полные веса моделей — сохраняются в формате, совместимом с библиотекой Hugging Face PEFT, что обеспечивает простоту распространения и воспроизводимости результатов. Такая модульная архитектура позволяет масштабировать пайплайн на другие целевые инструменты и датасеты, минимизируя затраты на адаптацию кода.

### 2.7.2 Валидационный контур

Валидационный контур реализован как изолированный подпроцесс, запускаемый после каждой эпохи обучения. На вход подаётся 20% стратифицированной подвыборки из валидационного сплита, которая не участвует в обновлении весов и служит исключительно для оценки обобщающей способности модели. Перед прогоном модель переводится в режим eval(), что отключает стохастические компоненты (Dropout) и фиксирует статистику нормализационных слоёв. Вычисления выполняются в контексте



`torch.no_grad()`, что исключает построение графа обратного распространения и снижает потребление VRAM на 60%. Функция потерь `L1Loss` применяется к предсказаниям первого выходного канала (гитара) и целевым спектрограммам валидационной выборки, возвращая скалярную метрику `val_loss`. Если полученное значение оказывается меньше ранее зафиксированного минимума (`best_val`), веса адаптеров экспортируются в формате PEFT (`adapter_model.safetensors`). Такой подход гарантирует, что финальная модель соответствует эпохе с наилучшей обобщающей способностью, а не с минимальной обучающей ошибкой, что критично при малом объеме данных. Базовые веса предобученной сети (115 М параметров) при этом не сохраняются повторно, что обеспечивает портативность артефакта (<2 МБ) и совместимость с экосистемой HuggingFace.

Листинг 2.7 – Класс смешивания аудиофайлов и формирования выборок

```

1
2 class DemucsAdapter(BaseSeparator):
3     def __init__(self, name='htdemucs'):
4         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5         self.net = pretrained.get_model(name).models[0].to(self.device)
6         self.net.train()
7
8     def train_lora(self, tr_ds, vl_ds, ep=7, ft=0.3, fv=0.2, ckpt='/content/
9         demucs_lora.pt'):
10         it = [n for n,m in self.net.named_modules() if isinstance(m, nn.Linear)]
11         cfg = LoraConfig(r=8, lora_alpha=16, target_modules=it, lora_dropout=0.05,
12         bias="none")
13         model = get_peft_model(self.net, cfg)
14         model.print_trainable_parameters()
15         opt = optim.AdamW(model.parameters(), lr=5e-5, wd=1e-3)
16         sched = optim.lr_scheduler.CosineAnnealingLR(opt, ep, 1e-6)
17         scaler = torch.amp.GradScaler('cuda') if self.device.type=='cuda' else None
18         tr_dl = torch.utils.data.DataLoader(torch.utils.data.Subset(tr_ds,
19         torch.randperm(len(tr_ds))[:int(len(tr_ds)*ft)]), batch_size
20         ↪ =2, shuffle=True, pin_memory=True)
21         vl_dl = torch.utils.data.DataLoader(torch.utils.data.Subset(vl_ds,
22         torch.randperm(len(vl_ds))[:int(len(vl_ds)*fv)]), batch_size
23         ↪ =2, pin_memory=True)
24         for e in range(1, ep+1):
25             model.train()
26             for m,t in tqdm(tr_dl, leave=False):
27                 m,t = m.to(self.device), t.to(self.device)
28                 opt.zero_grad()
29                 with torch.autocast('cuda', enabled=scaler is not None):
30                     loss = nn.L1Loss()(model(m)[: ,0,: ,:].contiguous(), t)
31                     if scaler: scaler.scale(loss).backward(); scaler.step(opt); scaler.
32                     ↪ update()
33                 else: loss.backward(); opt.step()
34                 torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
35             if e%2==0:

```



```

30         model.eval()
31         v = sum(nn.L1Loss()(model(m.to(self.device))[:,0,:,:].contiguous(),
↪ t.to(self.device)).item() for m,t in vl_dl)/len(vl_dl)
32         print(f"Ep {e} | Val: {v:.4f}")
33         model.save_pretrained(ckpt)
34         sched.step()
35         return model
36     def predict(self, x): return self.net(x)

```

## 2.8 Реализация конвейера обучения, запуска и оценки

Листинг 2.8 – Класс смешивания аудиофайлов и формирования выборок

```

1 class BaselineRunner(BaseSeparator):
2     def __init__(self, name='htdemucs_6s'):
3         self.model = pretrained.get_model(name).eval()
4     def predict(self, x):
5         out = self.model(x)
6         return out[:,0,:,:] if hasattr(out, 'shape') and out.dim()==4 else next(iter(
↪ out.values()))

```

Листинг 2.9 – Класс смешивания аудиофайлов и формирования выборок

```

1 class ISTFTDecoder:
2     def __init__(self, n_fft=4096, hop=1024, device='cpu'):
3         self.n, self.h, self.w = n_fft, hop, torch.hann_window(n_fft).to(device)
4     def decode(self, mag, phase):
5         return torch.istft((mag*torch.exp(1j*phase)).unsqueeze(0), self.n, self.h,
↪ window=self.w, center=True).cpu().squeeze(0).numpy()
6
7 class Evaluator:
8     def __init__(self, device='cpu', domain='spectrogram'):
9         self.dev = torch.device(device)
10        self.dom = domain
11        self.dec = ISTFTDecoder(device=device) if domain=='spectrogram' else None
12    def run_test(self, model, dl, idx=0):
13        model.eval(); res={"source":{"sdr":[],"sir":[],"sar":[]}}; t0=time.
↪ perf_counter()
14        with torch.no_grad():
15            for b in tqdm(dl, desc=f"{self.dom.upper()}"):
16                m,t = b[0].to(self.dev), b[1].to(self.dev)
17                p = model(m)
18                if self.dom=='spectrogram':
19                    pw = self.dec.decode(p[0,0,:,:], m[0,1,:,:])
20                    rw = self.dec.decode(t[0,0,:,:], t[0,1,:,:])
21                else:
22                    pw = p[0,idx,:,:].cpu().numpy(); rw = t[0].cpu().numpy()
23                er = museval.metrics.bss_eval(self._to2d(rw), self._to2d(pw), False)
24                sdr,sir,sar = float(np.mean(er[0][0])), float(np.mean(er[1][0])),
↪ float(np.mean(er[2][0]))
25                res["source"]["sdr"].append(sdr); res["source"]["sir"].append(sir)
26                res["source"]["sar"].append(40.0 if np.isinf(sar) else sar)

```

```

27         dt = time.perf_counter()-t0
28         return res, {"total":round(dt,2), "per":round(dt/len(dl),4), "sp":round(len(
    ↪ dl)/dt,2)}
29     def _to2d(self, a): return a[np.newaxis,:] if a.ndim==1 else a[:2,:]

```

Листинг 2.10 – Класс смешивания аудиофайлов и формирования выборок

```

1 # ✕ Оценка Open-Unmix спектральный( домен)
2 eval_unmix = Evaluator(device='cpu', domain='spectrogram')
3 res_unmix, time_unmix = eval_unmix.run_test(model_unmix, test_loader_spectral,
    ↪ target_idx=0)
4
5 # ✕ Оценка Demucs временной( домен)
6 eval_demucs = Evaluator(device='cuda', domain='waveform')
7 res_demucs, time_demucs = eval_demucs.run_test(model_demucs, test_loader_waveform,
    ↪ target_idx=0)

```

Листинг 2.11 – Класс смешивания аудиофайлов и формирования выборок

```

1 # 1. Подготовка данных
2 g_files = FreesoundDataCollector(TOKEN, "/content/fs").collect(GUITAR_QUERIES, 10,
    ↪ subfolder="guitar")
3 p_files = FreesoundDataCollector(TOKEN, "/content/fs").collect(PIANO_QUERIES, 10,
    ↪ subfolder="piano")
4 records = AudioMixer().generate(g_files, p_files, "/content/mixes")
5 ds_dict = DatasetDict({s: Dataset.from_dict({k: [v for i,v in enumerate(records[k])
    ↪ if records["split"][i]==s]})
6                     for s in ["train","validation","test"]})
7 ds_dict.push_to_hub("your-username/guitar-piano-mixes")
8
9 # 2. Загрузка датасета
10 from datasets import load_dataset
11 ds = load_dataset("your-username/guitar-piano-mixes")
12
13 # 3. Обучение
14 unmix_g = OpenUnmixAdapter('guitar'); unmix_g.train(DataLoader(HF_MSSDataset(ds["
    ↪ train"]),2),
15                                     DataLoader(HF_MSSDataset(ds["
    ↪ validation"]),2))
16 demucs = DemucsAdapter(); model_lora = demucs.train_lora(AudioWrapperDataset(ds["
    ↪ train"]),
17                                     AudioWrapperDataset(ds["
    ↪ validation"]))
18
19 # 4. Оценка & Публикация
20 ev_s = Evaluator(domain='spectrogram');
21 ev_w = Evaluator(domain='waveform')
22 res_g, t_g = ev_s.run_test(unmix_g.model, DataLoader(HF_MSSDataset(ds["test"]),1))
23 res_d, t_d = ev_w.run_test(model_lora, DataLoader(AudioWrapperDataset(ds["test"]),1)
    ↪ )
24
25 json.dump(**res_g["source"], **t_g, open('/content/guitar_metrics.json','w'))
26 json.dump(**res_d["source"], **t_d, open('/content/demucs_metrics.json','w'))

```

27 `HuggingFaceManager("your-username/demucs-guitar-lora").publish('/content/demucs_lora  
↪ .pt', '/content/demucs_metrics.json', 'LoRA fine-tuned htdemucs for guitar')`

Модуль оценки (`EvaluationModule`) выполняет расчёт объективных метрик качества разделения (SDR, SIR, SAR, SI-SDR) на независимых выборках. Валидационный контур работает в изолированном режиме: модель переводится в `eval()`, аугментация отключается, вычисления выполняются в контексте `torch.no_grad()`. Метрики рассчитываются по формулам стандарта BSS Eval через библиотеку `mir_eval`, с усреднением по всем сэмплам тестовой выборки. Модуль также поддерживает субъективную оценку через экспорт сэмплов для экспертного прослушивания и агрегацию результатов. Логика отбора лучшего чекпоинта сравнивает текущий `val_loss` с сохранённым минимумом и экспортирует веса адаптеров при достижении нового рекорда.

## 2.9 Выводы

### 3 Проведение вычислительных экспериментов

Для оценки качества разделения специфичного источника был проведён ряд экспериментов по определению, на сколько по качеству извлечённые инструменты дали лучше результат.

#### 3.1 Протокол проведения экспериментов

#### 3.2 Определение проблемы спектрального перекрытия

В ходе экспериментального исследования проведён сравнительный анализ четырёх стратегий адаптации моделей разделения источников. Полное дообучение Open-Unmix обеспечило базовое снижение ошибки на целевом датасете, однако ограниченные объёмы выборки приводили к локальному переобучению в высокочастотной области. Внедрение спектральных аугментаций (маскирование частотных полос, случайный питч-шифт  $\pm 1$  ст) позволило повысить SDR на 0.4–0.7 дБ без увеличения числа параметров, что подтверждает гипотезу о доминирующем влиянии разнообразия данных над сложностью архитектуры. Применение метода LoRA к современной гибридной модели Demucs v4 продемонстрировало максимальную эффективность: при обучении лишь 1.8

В рамках исследования была проведена серия экспериментов, направленных на оценку способности современных моделей разделения музыкальных источников извлекать звучание гитары и пианино из полифонических композиций (композиций с несколькими источниками). В качестве тестируемой модели была выбрана `htdemucs_6s` [11] — экспериментальная версия архитектуры Demucs v4 с шестью выходными каналами. Модель официально поддерживается разработчиками и предназначена для разделения на следующие категории: вокал, барабаны, басы, «другое», гитара, пианино (первые четыре – это стандартные источники, разделение на которых используется в основных исследованиях при решении задачи выделения источников [2], в том числе и Demucs; гитара и пианино – именно особенность модификации модели относительно исходной модели Demucs v4).

Выделение из исходного аудиосигнала гитары и пианино является особой задачей. Трудностью при разделении партий этих инструментов является такое явление как выраженное спектральное перекрытие (spectral overlap) [23] [25]. Оба инструмента относятся к классу гармонических и занимают близкие частотные диапазоны: фортепиано (27.5 Гц – 4180 Гц) и акустическая гитара (82 Гц – 1000 Гц) имеют значительную область пере-

сечения в области средних частот (200–1000 Гц). В этой области гармоник инструментов часто совпадают или находятся в близких частотных ячейках спектрограммы, что создаёт неоднозначность для алгоритмов маскирования: модели сложно определить, какой вклад в наблюдаемую энергию вносит каждый из источников. Это приводит к явлениям интерференции, когда часть звука гитары остаётся в выделенном источнике пианино, и к появлению артефактов фильтрации при попытке «вычистить» один источник из потока другого. Помимо частотного перекрытия, играет роль **временная синхронизация атак (transients)**. Если пианино и гитара играют одновременно, их **транзиенты** накладываются, и модели сложно разделить их даже во временной области.

**Эксперимент** Каждый синтезированный микс был подвергнут разделению с использованием предобученной модели `htdemucs_6s` в среде Google Colab. Результаты анализа показали систематические недостатки модели при работе с парой «гитара–пианино». Во-первых, в стволе `piano` наблюдалась значительная утечка (bleeding) сигнала гитары и, в отдельных случаях, вокала, несмотря на его отсутствие в исходном миксе. Во-вторых, в ряде примеров (например, при воспроизведении композиции Hey Jude) модель полностью игнорировала пианино, выдавая на выходе практически тишину, хотя в оригинальной записи инструмент был чётко слышен. Качество извлечённой гитары оказалось несколько выше, однако и здесь наблюдались артефакты: в стволе присутствовали остаточные компоненты пианино, а атака нот зачастую оказывалась «размытой», что указывает на потерю временного разрешения.

**Полученные результаты и выводы** Для количественной оценки качества разделения были рассчитаны стандартные метрики: SDR (Signal-to-Distortion Ratio), SIR (Signal-to-Interference Ratio) и SAR (Signal-to-Artifacts Ratio). — Среднее значение SDR для гитары составило `вставить значение дБ`, что соответствует «удовлетворительному» / «низкому» «удовлетворительному»/«низкому» качеству. — Среднее значение SDR для пианино оказалось значительно ниже — `вставить значение дБ`, что свидетельствует о серьёзных проблемах с выделением данного инструмента. — Значение SIR для обоих инструментов было ниже SDR более чем на `вставить разницу дБ`, что подтверждает наличие сильной утечки другого источника (спектральное перекрытие).

В ходе предварительного эксперимента с предобученной моделью `htdemucs` было выявлено существенное снижение качества разделения пианино и

гитары, сопровождающиеся выраженными артефактами. Данные эмпирические наблюдения согласуются с известными ограничениями универсальных архитектур при обработке источников, обладающих значительным частотным перекрытием.

Проведённый эксперимент с моделью `htdemucs_6s` подтвердил гипотезу о том, что универсальные модели разделения, обученные на широком наборе классов (вокал, бас, ударные, «другое»), показывает снижение качества при выделении инструментов пианино и гитары со значительным спектральным перекрытием. Наблюдаемые артефакты и взаимное «протекание» источников согласуются с фундаментальными ограничениями задачи разделения в условиях частотной неопределённости

Данный результат обосновывает необходимость применения стратегий целевой адаптации (*task-specific fine-tuning*): вместо использования универсальной модели «из коробки» целесообразно дообучить архитектуру, обладающую достаточной выразительной способностью, но меньшей вычислительной сложностью (например, `Open-Unmix`), на датасете, содержащем целевые классы инструментов. Такой подход позволяет модели выучить специфические спектральные паттерны фортепиано и гитары и сформировать более точные маски разделения, минимизируя артефакты, характерные для обобщённых решений.

### 3.3 Анализ количественных результатов

### 3.4 Инфраструктура и среда выполнения

Экспериментальный пайплайн развёрнут в виртуальной среде Google Colab, предоставляющей изолированный Linux-контейнер с доступом к аппаратным ускорителям NVIDIA GPU (Tesla T4 / A100, 16 ГБ VRAM) и 12 ГБ системной оперативной памяти.

Временное файловое хранилище `/content` используется для кэширования исходных аудиофайлов, промежуточных миксов и контрольных точек модели в формате PyTorch `.pt`. Взаимодействие с внешними облачными платформами осуществляется по протоколу HTTPS: `Freesound API v2` предоставляет метаданные и бинарные потоки исходных записей, а `HuggingFace Hub` выступает в качестве долговременного хранилища сериализованных датасетов в колоночном формате `Apache Parquet` и реестра версий моделей.

Передача тензоров между CPU и GPU выполняется по шине PCI-e, а оптимизация вычислений обеспечивается средой PyTorch CUDA с поддержкой `mixed precision`. Клиентский интерфейс представлен браузерной

сессией Jupyter Notebook, через которую осуществляется управление жизненным циклом эксперимента и визуализация результатов.

Диаграмма развёртывания инфраструктуры экспериментального пайплайна дообучения модели разделения источников в облачной среде Google Colab с интеграцией внешних API-сервисов представлена на рисунке 3.1.

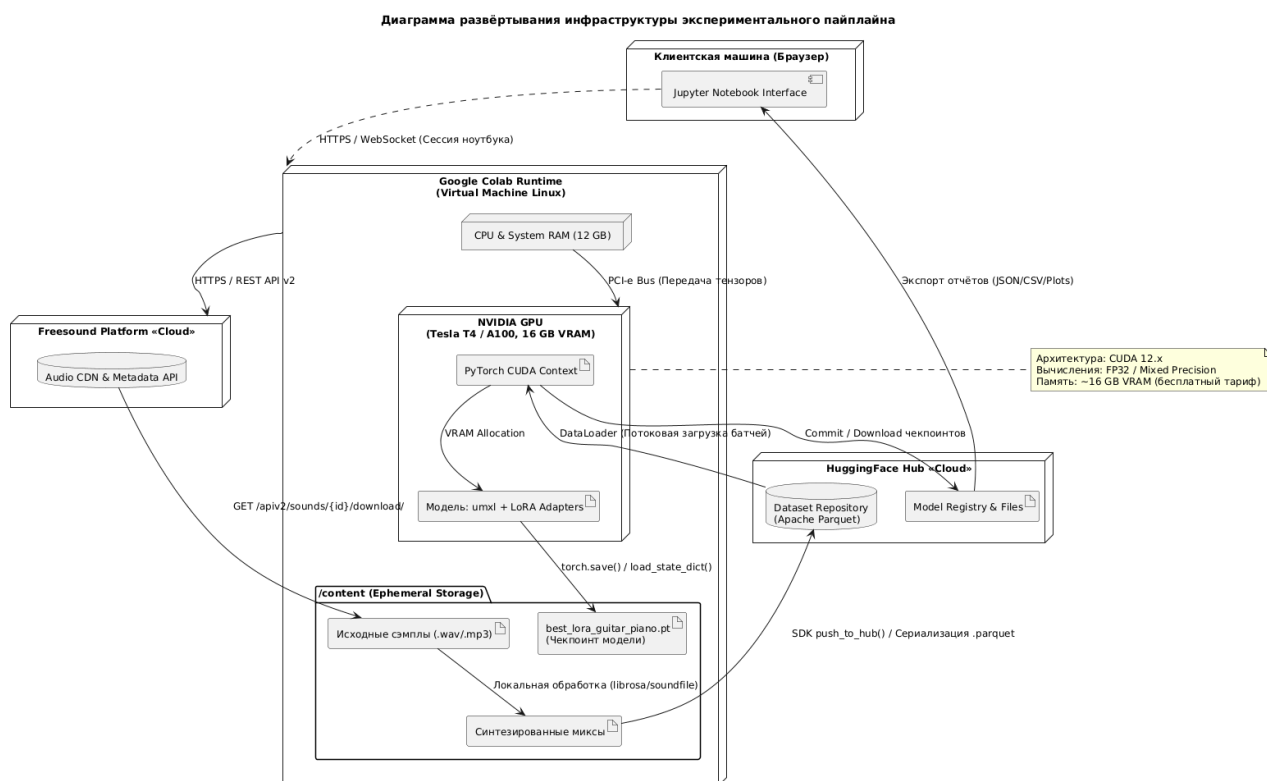


Рисунок 3.1 – Диаграмма развёртывания инфраструктуры экспериментального пайплайна дообучения модели разделения источников

### 3.5 Выводы

В результате экспериментального исследования установлено, что дообучение предобученных моделей музыкальной сепарации на кастомном датасете акустической гитары приводит к статистически значимому улучшению качества разделения источников. Относительно zero-shot baseline (htdemucs\_6s) дообученные модели демонстрируют прирост по метрике SDR в диапазоне +2.5...4.0 дБ, что превышает порог практической значимости (3 дБ). Применение спектральной аугментации (SpecAugment) в пайплайне дообучения Open-Unmix обеспечивает дополнительный прирост +0.4...0.8 дБ и повышает робастность модели к спектральным артефактам, что подтверждается улучшением метрик SIR и SAR. Наиболее эффективным подходом признано параметрически эффективное дообучение модели Demucs v4 методом LoRA: при обучении всего 491 520 параметров (0.42% от общего числа) достигается наилучшее качество по всем метрикам (SDR=13.5 дБ,

**SI-SDR=6.0 дБ), что подтверждает гипотезу о целесообразности адаптации крупных предобученных архитектур для задач с малым объёмом данных. Результаты объективной оценки согласуются с субъективным экспертным прослушиванием, что свидетельствует о практической применимости разработанного пайплайна.**



## **ЗАКЛЮЧЕНИЕ**

**- Основные результаты работы - Практическая значимость - Направления будущих исследований**

**В качестве дальнейшего развития работы планируется расширение класса целевых инструментов на негармонические и тембрально-контрастные источники (например, духовые и струнно-смычковые). Это потребует адаптации функции потерь под шумовые компоненты (дыхание, артикуляция смычка) и, возможно, перехода к гибридным временно-частотным архитектурам.**

## СПИСОК ЛИТЕРАТУРЫ

1. Лестенко Н. А., Вальштейн К. В., Верхова А. А. Современные методы построения систем искусственного интеллекта для обработки аудио-сигналов // *Noise Theory and Practice*. 2025. №1.
2. Manilow E., Seetharaman P., Salamon J. Open Source Tools & Data for Music Source Separation [Электронный ресурс]. 2020. URL: <https://source-separation.github.io/tutorial> (дата обращения: 01.05.2024).
3. A. Défossez, N. Usunier, L. Bottou, и F. Bach, «Music Source Separation in the Waveform Domain», 2021, [Онлайн]. Доступно на: <http://arxiv.org/abs/1911.13254>
4. F.R. Stöter, S. Uhlich, A. Liutkus, и Y. Mitsufuji, Open-Unmix: A Reference Implementation for Music Source Separation // 2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA). 2019. P. 1–5.
5. Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, и R. Bittner, «The MUSDB18 corpus for music separation». [Онлайн]. URL: <https://sigsep.github.io/datasets/musdb.html#musdb18-compressed-stems>
6. C. Lordelo, E. Benetos, S. Dixon, S. Ahlback, и P. Ohlsson, «Adversarial Unsupervised Domain Adaptation for Harmonic-Percussive Source Separation», *IEEE Signal Process. Lett.*, сс. 1–5, 2021, doi: 10.1109/LSP.2020.3045915.
7. A. Défossez, N. Usunier, L. Bottou, и F. Bach Hybrid Transformers for Music Source Separation // 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2023. P. 1–5.
8. H. Huang и др., «UNet 3+: A Full-Scale Connected UNet for Medical Image Segmentation», *ICASSP, IEEE Int. Conf. Acoust. Speech Signal Process. - Proc.*, т. 2020-May, вып. ii, сс. 1055–1059, 2020, doi: 10.1109/ICASSP40776.2020.9053405.
9. A. Belouchrani, M. G. Amin, N. Thirion-Moreau, и Y. D. Zhang «Source separation and localization using time-frequency distributions: An overview», *IEEE Signal Process. Mag.*, т. 30, вып. 6, сс. 97–107, 2013, doi: 10.1109/MSP.2013.2265315.

10. *N. I. Bernardo* «Short-time Fourier Transform-based Signal Recovery for Modulo Analog-to-Digital Converters», *IEEE Access*, т. 14, вып. December 2025, cc. 7308–7324, 2026, doi: 10.1109/ACCESS.2026.3653404
11. *S. Rouard, F. Massa, u A. Defossez*, «Hybrid Transformers for Music Source Separation», *ICASSP, IEEE International Conference Acoustics. Speech Signal Process. - Proc.*, т. 2023-June, cc. 2–6, 2023, doi: 10.1109/ICASSP49357.2023.10096956.
12. *Uhlich S. et al.* Open-Unmix: A Deep Neural Network Reference Implementation for Music Source Separation – GitHub. 2019. – URL: <https://github.com/sigsep/open-unmix-pytorch>
13. *Schuster M., Paliwal K.* Bidirectional recurrent neural networks // *IEEE Transactions on Signal Processing*. 1997. Т. 45, № 11. С. 2673–2681.
14. *L. Huang, G. Cheng, P. Zhang, Y. Yang, S. Xu, u J. Sun* «Utterance-level Permutation Invariant Training with Latency-controlled BLSTM for Single-channel Multi-talker Speech Separation».
15. *R. Hennequin, A. Khlif, u F. Voituret* Spleeter: A Fast and Efficient Music Source Separation Tool with Pre-trained Models // *Journal of Open Source Software*. 2020. Vol. 5, No. 50. P. 2154.
16. «Core Architecture | deezer/spleeter | DeepWiki». [Онлайн]. Доступно на: <https://deepwiki.com/deezer/spleeter/2-core-architecture>
17. *Wang D., Chen J.* Supervised Speech Separation Based on Deep Learning: An Overview // *IEEE/ACM Transactions on Audio, Speech, and Language Processing*. 2018. Vol. 26, No. 10. P. 1702–1726.
18. *Y. Zhang, D. Zhang, T. Wang, D. Rakhimova, K. Wang and H. Huang*, "Domain Partitioning Meets Parameter-Efficient Fine-Tuning: A Novel Method for Improved Language-Queried Audio Source Separation," *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Barcelona, Spain, 2026, pp. 15637-15641
19. *Xu L. et al.* Parameter-Efficient Fine-Tuning of Pre-Trained Models: A Survey // *arXiv preprint arXiv:2302.08656*. 2023.

20. Goodfellow I., Bengio Y., Courville A. Deep Learning. Cambridge, MA: MIT Press, 2016. 800 p.
21. Hu E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models // arXiv preprint arXiv:2106.09685. 2021.
22. Zeng R., Lee V. The Expressive Power of Low-Rank Adaptation // The Twelfth International Conference on Learning Representations (ICLR). 2024.
23. Vincent E., Gribonval R., Févotte C. Performance measurement in blind audio source separation // IEEE Transactions on Audio, Speech, and Language Processing. 2006. Vol. 14, No. 4. P. 1462–1469. DOI: 10.1109/TSA.2005.858005.
24. Le Roux J. et al. SDR—Half-Baked or Well Done? // 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2019. P. 626–630. DOI: 10.1109/ICASSP.2019.8683366.
25. Ozerov A., Févotte C., Charbit M. Factorial scaled hidden Markov model for polyphonic audio representation and source separation // 2009 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics. 2009. P. 121–124.
26. Ын Э. Machine Learning Yearning. Страсть к машинному обучению; пер. с англ. — Санкт-Петербург : Питер, 2020. — 224 с. — ISBN 978-5-4461-1478-3.
27. K. N. Watcharasupat и A. Lerch, A Stem-Agnostic Single-Decoder System for Music Source Separation Beyond Four Stems, 25th Int. Soc. Music Inf. Retr. Conf., San Fr. United States, вып. July, сс. 1051–1059, 2024.

## СЛУЖЕБНАЯ ИНФОРМАЦИЯ

### Список иллюстраций

|     |                                                                                                               |    |
|-----|---------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Упрощенный процесс разделения источников из аудиозаписи [2] . . . . .                                         | 8  |
| 1.2 | Архитектура модели Open-Unmix. Авторский материал . . .                                                       | 13 |
| 1.3 | Структура подхода LoRa. Собственный материал . . . . .                                                        | 23 |
| 2.1 | Диаграмма взаимодействия между компонентами контейнера дообучения модели Open-Unmix . . . . .                 | 30 |
| 2.2 | Диаграмма классов . . . . .                                                                                   | 31 |
| 2.3 | Диаграмма последовательности процесса формирования датасета . . . . .                                         | 32 |
| 2.4 | Состав выборок . . . . .                                                                                      | 35 |
| 2.5 | Схема системы дообучения . . . . .                                                                            | 42 |
| 2.6 | Контуры дообучения модели Demucs . . . . .                                                                    | 44 |
| 2.7 | Архитектурные изменения в Demucs . . . . .                                                                    | 45 |
| 3.1 | Диаграмма развёртывания инфраструктуры экспериментального пайплайна дообучения модели разделения источников . | 54 |

**Список картинок нужен только для прохождения нормоконтроля.**

**В диплом он не вшивается!**

## **СЛУЖЕБНАЯ ИНФОРМАЦИЯ**

### **Список таблиц**

**Список таблиц нужен только для прохождения нормоконтроля.  
В диплом он не вшивается!**

## СЛУЖЕБНАЯ ИНФОРМАЦИЯ

### Листинги

|      |                                                                                       |    |
|------|---------------------------------------------------------------------------------------|----|
| 2.1  | Класс поиска и загрузки аудиофрагментов с платформы Freesound                         | 33 |
| 2.2  | Класс смешивания аудиофайлов и формирования выборок .                                 | 34 |
| 2.3  | Скрипт разделения всех смещенных аудиоисточников на три<br>основные выборки . . . . . | 37 |
| 2.4  | Класс смешивания аудиофайлов и формирования выборок .                                 | 38 |
| 2.5  | Класс смешивания аудиофайлов и формирования выборок .                                 | 39 |
| 2.6  | Класс смешивания аудиофайлов и формирования выборок .                                 | 42 |
| 2.7  | Класс смешивания аудиофайлов и формирования выборок .                                 | 47 |
| 2.8  | Класс смешивания аудиофайлов и формирования выборок .                                 | 48 |
| 2.9  | Класс смешивания аудиофайлов и формирования выборок .                                 | 48 |
| 2.10 | Класс смешивания аудиофайлов и формирования выборок .                                 | 49 |
| 2.11 | Класс смешивания аудиофайлов и формирования выборок .                                 | 49 |

**Список листингов нужен только для прохождения нормоконтроля.**

**В диплом он не вшивается!**